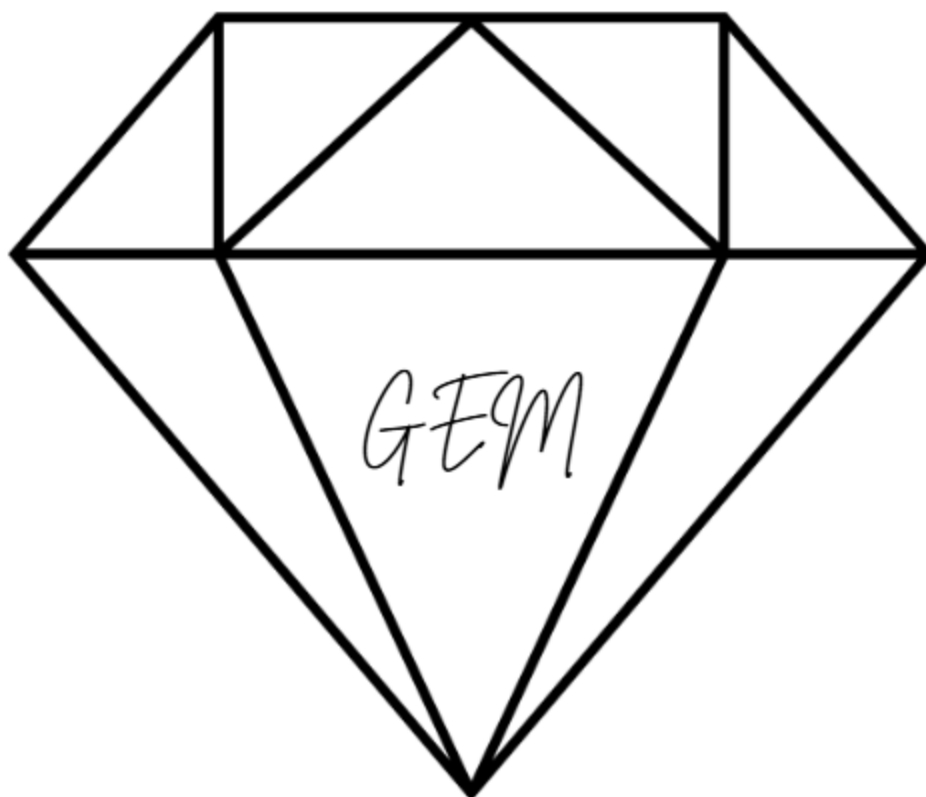




Guardian of Electronic Machines

Genevieve McNeil

03/30/2025

Lab 02 - Password Cracking

Executive Summary

This password cracking lab consists of three main parts. These parts include: Part 1: Introduction + Updating Kali Linux, Part 2: Cracking the Password Files, and finally, Part 3: Exploring Additional Password Cracking Tools. These three sections provide students with hands-on experience with password cracking, which is important for daily cybersecurity operations and future careers.

Through a variety of exercises, students get a deeper understanding of password cracking with tools such as Hashcat and Johnnny to crack passwords with the SAM PWDUMPed Passwords and shadow files. Supplementary tools such as Username_generator and CeWL were also utilized to enhance password cracking by generating custom wordlists and usernames. Most tools need a fair amount of time to do their job. As noted throughout the report, using these tools can be challenging, and thorough research and collaboration, are at times essential when completing labs with accuracy.

Table of Contents

Executive Summary	2
Part 1: Introduction + Updating Kali Linux.....	4
Part 2: Cracking the Password Files	9
Part 3: Exploring Additional Password Cracking Tools	13
Conclusion	16
Works Cited	17
Appendix	19

Part 1: Introduction + Updating Kali Linux

I installed Oracle VM VirtualBox Machine and created a Kali Linux Virtual Machine (VM) while I was taking a previous course last semester, IS 3033 (Operating Systems and Security). Therefore, I mainly needed to update the VM to complete this lab. However, I would like to note that I had some roadblocks when beginning the lab.

To begin the lab, I reviewed my Kali Linux VM. I made sure to check the CPU settings and other configurations as well. The machine was set up correctly for the most part, thankfully. This was because I had professor Elgamal assist with setting up the machine last semester (as it was my first time using a Kali Linux VM). As the instructions advised, I ensured that the VM had 50GB of hard drive space, 4cpu cores, and 8GB of RAM.

Afterwards, as I was using my laptop in class, I received a concerning message from Windows Defender. I reviewed the report that it had generated. There was a variety of notifications stating that the antivirus detected severe threats on my computer. I decided to scan my computer for any further threats. The threats included backdoors, Trojans, and much more. I then removed the threats from my computer. I would like to note that as I was reviewing the threat details, they indicated that they derived from my Kali Linux VM. So, I removed the ISO file from my computer. I was very concerned about these notifications, as this was the first time that my computer had any threats detected. I re-scanned my computer once again, and thankfully, there were no threats found. After attending the in person IS 6303 class on 03/19, Professor Bloodworth reviewed my Kali Linux VM and ensured the configurations were correct in order for me to continue the project.

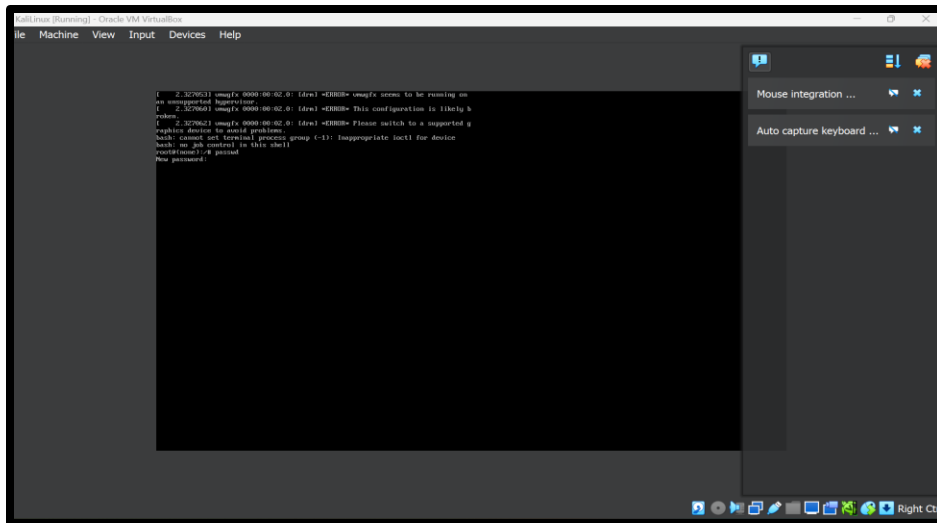


Figure 1: The figure above image includes a few errors I received when opening Kali Linux for the first time.

An additional roadblock I encountered was that I had not remembered my Kali Linux VM password from last semester. I decided to research online prior to making a new Kali Linux VM to save myself some time. I thankfully found an informative video and followed the steps. This is the video that assisted me in the process: [How to Reset Forgotten Password on Kali Linux](#).

I first started by taking note of my username for the Kali Linux VM. I then restarted the VM. As the machine booted, I was greeted with a small menu that prompted me to select one of these two options: “Kali GNU/Linux” or “Advanced options for Kali GNU/Linux.” I then selected: “Kali GNU/Linux” and entered “e” so that I can edit the grub entry (Figure 2). I used my cursor to navigate to “linux /boot/vm.....ro quiet splash”, also shown in Figure 2. At this point, I modified the boot parameters so I can reset the password and drop into the bash shell. I first edited “ro”, which means “read only” (which tells the kernel to boot the machine with a read-only file system). I edited “ro” to “rw” (which means “read write”), in order for me to interact with the file system and change the password to my machine (Conda). At the end of the same line, I added

menu: Hardening (for extra security), Metapackages (to install subsets of tools for specific needs), Network Repositories (to configure network repositories for sources of APT), Shell & Prompt (to configure the shell and command prompt), or Virtualization (additional configurations for virtual machines). I selected Metapackages and it promptly installed updates (as shown in Figure 5). Afterwards, it displayed the metapackages that I could choose to download as well as the metapackages that were previously downloaded.

I decided to download the metapackage “kali-tools-bluetooth.” I was then prompted to complete an update in regards to the metapackages. I decided not to, as I was more interested in seeing how this downloading process works within kali tweaks. To be transparent, I downloaded kali-tools-bluetooth mainly because the tools piqued my interest and I wanted to learn how it works for targeting bluetooth devices. The output after installing kali-tools-bluetooth included “The following additional packages will be installed: blue-hydra bluelog blueranger bluesnarfer bluez-test-scripts btscanner crackle” and much more.

As shown in Figure 7, I had top10, kali-linux-default, kali-linux-firmware, and kali-linux-headless downloaded. Top10 includes 10 of the most commonly used tools for pentesting in Kali Linux. I was interested in installing it since it provides a variety of useful tools for pentesting. Additionally, because it is a popular metapackage, I thought that it could be a valuable resource to utilize in the future. In general, I had downloaded kali-linux-default (default toolset for Kali), kali-linux-firmware (default firmware files for Kali), and kali-linux-headless (default headless tools for Kali) because they are part of the default set of tools. I normally prefer to download and update tools or software that

are part of default packages for my peace of mind, as I believe they are essential.

Because a lot of the tools in Kali Linux are resource-intensive, hardware performance plays an important role in their effectiveness. Password cracking relies on CPU processing power, memory (RAM), and storage speed to operate successfully.

The Central Processing Units (CPUs) are at the heart of any computing device. Its processing power is important, as it directly influences the speed at which a password cracking tool can operate and attempt guesses ("How Does Hardware Affect the Efficiency of Password Cracking?"). Furthermore, the CPU's speed determines the number of attempts it can make per second, which has a significant impact on the success of brute-force and dictionary attacks ("How Does Hardware Affect the Efficiency of Password Cracking?"). Multiple cores allow parallel processing, which in turn, speeds up brute-force attacks because it runs several hash calculations simultaneously ("How Does Hardware Affect the Efficiency of Password Cracking?").

Memory (RAM) and storage also play an important role in password cracking. The amount of available memory and the speed of storage impact the performance of password cracking tools, particularly in handling large dictionaries ("How Does Hardware Affect the Efficiency of Password Cracking?"). When using wordlists, the larger the dataset, the more memory is needed to load and access that data. Insufficient memory can slow down the cracking process, leading to longer password recovery times ("How Does Hardware Affect the Efficiency of Password Cracking?"). High-speed storage devices, such as SSDs, improve efficiency by enabling quicker reading and writing of large hash databases or wordlists ("How Does Hardware Affect the Efficiency of Password Cracking?"). RAM is helpful during dictionary attacks through

storing more of the wordlist in memory, reducing the need for the system to read from the disk as often (“How Does Hardware Affect the Efficiency of Password Cracking?”).

Part 2: Cracking the Password Files

To start the process for password cracking, you will need to determine the hashing algorithm or encryption method used to store the passwords (ie: MD5, SHA-256, etc.). This is important because knowing the method used to store the passwords determines how the user will go about cracking it. Different algorithms need different techniques to crack their passwords. For Hashcat, I needed to specify MD5 (“-m 0”), while Johnny attempts to automatically detect the hashing method in most cases.

Johnny is “an offline password cracking tool that was developed in 1996 by Openwall Project” (Moyle). It supports a variety of password formats. Johnny cracks passwords through replicating the password-handling process of a target storage system (Moyle). This allows it to attempt multiple password values (guesses) without relying on the system's password validation (Moyle).

Hashcat takes password hashes as input and then uses various attack modes and masking techniques to create password guesses (Goplani). The generated guesses are hashed in the same format as the input hash and then compared to find a match (Goplani). Although Hashcat mainly uses GPUs for parallel processing, it can also utilize the CPUs for cracking (Goplani). Features such as Hashcat rules and mask attacks help create intelligent password mutations, making password cracking much more efficient (Goplani). I chose to utilize Hashcat and Johnny for my password cracking attempts.

In this section of the lab, I used two separate files within a password files zip folder containing password files, as noted in the instructions. I opened my VM and logged into <https://utsa.instructure.com/>. The two files were SAM PWDUMPed Passwords and shadow, so I went ahead and downloaded them to the VM. Once I unzipped the files, I placed them within a directory called "EXTRACTED." I also ensured that all the files that I created for the remainder of the lab were stored in that same folder.

As I mentioned earlier, I decided to use Johnny and Hashcat. Johnny is the GUI tool for John the Ripper, and in my opinion, it is an easy tool to use. However, it requires a lot of time to crack passwords. As shown in Figure 8, I entered "sudo apt install kali-linux-passwords -y" to download Hashcat, John the Ripper, and Hydra but it did not work. So, I decided to update the package list by inputting "sudo apt update" After updating, I tried "sudo apt install kali-linux-passwords -y" but it did not work again (Figure 9). "Kali-linux-passwords" was not listed in the output as a package, so I inputted "sudo apt install kali-linux-default -y" to install Kali tools (which includes John the Ripper and Hydra). This update took about 20 minutes, as it was downloading and unpacking a large number of tools (Figure 10). Afterwards, I entered "sudo apt install kali-linux-default -y" once again (Figure 11). I then decided to download Johnny as I still did not see it on the VM. I was able to successfully download it and open it on my VM. To start using Johnny, I opened the tool, selected "Open password file", "open password file PASSWD format", selected the file that I wanted to use, pressed "open", and then hit "start attack". I followed these steps for both the SAM PWDUMPed Passwords and shadow unzipped files within Johnny.

As shown in Figure 12, I began by running the two unzipped files in Johnny. I allowed Johnny to run for well over 4 hours and received error codes such as: “[watchdog: BUG: soft lockup” on CPU 0, 1, and 3, as shown in Figure 13. Soft lockups usually occur when the CPU is stuck in a loop and does not respond to other system tasks for a long period of time (“Watchdog: BUG: Soft Lockup”). To mitigate this issue, I decided to save the VM and then restart it. I changed the CPU from 4 to 6 in hopes this process would be a bit quicker. I started Johnny again and let it run for 6 more hours. Figure 14 displays the number of passwords that were cracked afterwards. 3/13 passwords were cracked from the Shadow file, while 12/23 were cracked for the SAM file. For the SAM file, the Administrator and Default passwords were “P@SSWORD”, Edgar’s password was “N3VERMORE”, Flower’s password was “CHRY4NTH3MUM”, Help assistant’s password was “???????4GDPBC1”, Kanye’s password was “000000”, and WoodyAllen’s was “HEYWOOD1211935”. For the shadow file, the password for root was “toor”, the password for PswDn00b was “1234”, Juliet’s password was “romeo”, and State’s password was Colorado. Figures 15 and 16 include the cracked passwords from running Johnny on the SAM PWDUMPed Passwords (right) and shadow (left) files. I would like to note that a lot of the users’ passwords are related to their names. For example, Colorado is a state, a “CHRY4NTH3MUM” (chrysanthemum) is a flower, and so on. I then decided to let the Virtual Machine run overnight in hopes to crack the rest of the passwords with Johnny. I awoke to another error message once again and restarted, as shown in Figure 17 and Figure 18. I created two notes containing the cracked passcodes in “SAM PWDUMPed Passwords cracked Johnny.txt” and “shadow cracked Johnny.txt”, as shown in Figure 19. I then

decided to move on and use Hashcat next, as I had spent a lot of time using Johnny and was excited to start using Hashcat for the first time.

To use Hashcat, I used the same two unzipped files SAM PWDUMPed Passwords and shadow within the "EXTRACTED" folder. In the command line, I inputted "ls /usr/share/wordlists" to see if I had "rockyou.txt.gz" - which I did. I unzipped the rockyou.txt.gz by inputting "sudo gzip -d /usr/share/wordlists/rockyou.txt.gz", which extracted the file as "rockyou.txt" in the /usr/share/wordlists/ directory. I then updated and downloaded the SecLists package (Figure 21). Initially, the output suggested that I successfully cracked 5/13 passwords from the shadow file by running "hashcat -m 1800 -a 0 shadow /usr/share/wordlists/rockyou.txt" (Figure 22 and 23). This command instructed Hashcat to take every password in "rockyou.txt", hash it using the SHA-512 Crypt algorithm, and compare it to the hashes within the shadow file. The "-m 1800" flag refers to the SHA-512 Crypt hashes, while "-a 0" indicates a dictionary (straight) attack. As shown in Figure 24, I received an error for using the original shadow file, meaning that I had not cracked 5 passwords as I initially thought. I then created a new file called "shadow hashes only" and removed the usernames and any extra words/symbols that were not part of the hash. I then re-ran the command "hashcat -m 1800 -a 0 "shadow hashes only.txt" /usr/share/wordlists/rockyou.txt". As shown in Figure 26, I was able to successfully crack 4 passwords within the shadow file. I then inputted "hashcat --show -m 1800 "shadow hashes only.txt" > "shadow cracked hashcat.txt", which redirected the passwords into a file called "shadow cracked hashcat.txt" (Figure 27). In Figure 28, you can view the cracked passwords from the shadow file.

I then moved on to use Hashcat to crack the SAM password File. I did run into a bit of confusion here, as I had error codes whilst finding rockyou.txt in the correct directory. However, I was able to figure it out. I then entered "hashcat -m 1000 -a 0 "SAM hashes only.txt /usr/share/wordlists/rockyou.txt" to initiate Hashcat on the SAM file. This command is used to attempt cracking passwords by comparing the hashes from the SAM file against the RockYou wordlist. Also, "-m 1000" specifies NTLM hashmode. As shown in Figures 31 and 32, I was only able to crack 2 passwords on the "SAM hashes only.txt" file using Hashcat. The passwords I cracked were "000000" (which I assume belongs to Kanye) and "P@ssw0rd" (potentially a default or administrator password). I had Hashcat complete a dictionary (straight) attack ("a -0"). I believe I was also able to crack those 2 passwords because they were much simpler compared to the other cracked passwords. I came to the realization a few days later that it was likely because I would need to execute a brute-force attack instead.

I personally believe that Johnny was a more effective tool because I was able to crack more passwords with it compared to Hashcat. Johnny's automatic detection of hash types most likely contributed to its success in cracking more passwords as well. I saved the passwords I cracked using Hashcat on the SAM and Shadow files on the VM, which can be seen in Figure 33.

Part 3: Exploring Additional Password Cracking Tools

CeWL is a "A custom wordlist generator... that crawls a specific URL to a defined depth and returns a list of keywords" (Ch and el). Password crackers such as John the Ripper, Medusa, and Wfuzz can use CeWL to crack the passwords. Additionally, CeWL has an associated command-line app named FAB, which "uses the same

metadata extraction techniques to generate author/producer lists from already downloaded files using information extraction algorithms like CeWL” (Ch and el).

It is noteworthy to mention that CeWL is preinstalled with Kali Linux. It can collect words and phrases from a target page because of its useful features (Ch and el). It is a program that can scrape a web server of any website within a short amount of time (Ch and el). Overall, it pulls information from websites to create wordlists. It is helpful with password cracking because it incorporates terms that it pulled from the websites it is targeting.

By entering “cewl -help” in the command line, I was able to learn more about how the tool functions. The four options I would like to mention are:

- “-d”: The depth to spider to (the default is 2). In my opinion, this is useful because deeper crawling means the potential to collect more relevant words that can be used for password cracking.
- “-m”: Minimum word length (the default is 3). In my opinion, this can remove any short words that are useless for password cracking. It is helpful because users can set it to a different number, if desired.
- “-x”: Maximum word length (no default set). I noticed that most of the passwords I cracked were simple and not too lengthy. I believe that this limit ensures the list is not too cluttered with long words that are most likely not passwords.
- “-w”: Writes the output to the file (that users can specify). This is essential because tools like Hashcat or John the Ripper use (and need) a wordlist file.

As I will mention later in this section, I utilized `username_generator.py`. The username generator essentially creates potential usernames based on the input data that is normally received from CeWL's extracted words. By using CeWL with `username_generator`, it provides a more targeted approach to password cracking. As noted previously, CeWL collects words from a website of the user's choice which could include names, emails, or even terms connected to the target. `Username_generator` creates potential usernames based on that information. In my opinion, this makes the process much easier because users can then generate wordlists that match the target's potential credentials. When combined with tools like Johnny and Hashcat, wordlists can be used for dictionary attacks to crack hashed passwords efficiently. CeWL collects context-specific words, and `username_generator` helps generate potential usernames. I believe that this improves the chances of successfully cracking passwords.

As shown in Figure 34, I inputted the code `cewl -d 2 -m 5 -w custom.txt https://education.utsa.edu/faculty/`. The website is for the faculty and staff for College of Education and Human Development department at UTSA. The command `"-w custom.txt"` saves the words that were scraped into the `"custom.txt"` file. Also, `"-m 5"` extracts the words that are 5 or more characters. Finally, `"-d 2"` requests to only follow links that are from the main page all the way to the subpages. After running the command, I entered `"cat"` to view the output, which showed a large number of extracted words from the website, but no names. It was at this point that I contacted my classmate to assist me, as I was receiving errors. I inputted `cewl -- email -n -w emails.txt https://education.utsa.edu/faculty/` in order to pull the emails from the website (Figure 36). Then, I entered the command `awk -F '@.' '{print $1, $2}' emails.txt > custom.txt`

to extract usernames from emails.txt and save them in the new custom.txt file (Figure 37). I entered “python3 --version” and the results showed that I am running Python 3.13.2. Finally, I ran “python3 username_generator.py -w custom.txt > faculty_user_names.lst”, which generated a list of potential usernames. (Figure 38).

Conclusion

In conclusion, this lab was challenging yet rewarding. I encountered several issues which include: a “virus scare”, login issues with Kali Linux, and other difficulties throughout. While I did not expect this lab to be as challenging and demanding as it was, I view it as a valuable experience. I received a thorough understanding of the timeframe needed to complete password cracking, utilized Hashcat and CeWL for the first time, and successfully cracked passwords. The challenges of this lab teach the value of patience, research skills, and collaboration. I look forward to completing more labs throughout my education in UTSA and am grateful for the experience I had.

Works Cited

Ch, Raj, and el. "A Detailed Guide on Cewl." *Hacking Articles*, 7 Apr. 2022, www.hackingarticles.in/a-detailed-guide-on-cewl/. Accessed 26 Mar. 2025.

Conda. "How to Reset Forgotten Password on Kali Linux." *YouTube*, 18 Oct. 2021, www.youtube.com/watch?v=7a_SB4xzoY8. Accessed 21 Mar. 2025.

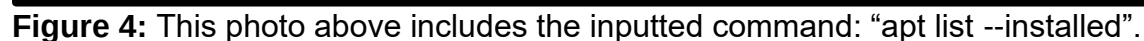
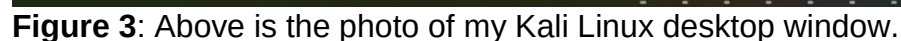
Goplani, Bhavik. "Hashcat vs John the Ripper: A Comparative Benchmarking of Password Cracking Tools 🚀." *DEV Community*, 29 Dec. 2023, dev.to/bhavikgoplani/hashcat-vs-john-the-ripper-a-comparative-benchmarking-of-password-cracking-tools-26a4. Accessed 29 Mar. 2025.

"How Does Hardware Affect the Efficiency of Password Cracking? – Cyberly." *Cyberly.org*, 24 Nov. 2024, www.cyberly.org/en/how-does-hardware-affect-the-efficiency-of-password-cracking/index.html?utm_source=chatgpt.com. Accessed 29 Mar. 2025.

Moyle, Ed. "How to Use the John the Ripper Password Cracker." *TechTarget*, 13 Apr. 2023, www.techtarget.com/searchsecurity/tutorial/How-to-use-the-John-the-Ripper-password-cracker. Accessed 29 Mar. 2025.

"Watchdog: BUG: Soft Lockup - CPU#0 Stuck When Booting (Arch) Linux on MINISFORUM N4020." *Unix & Linux Stack Exchange*, 8 Sept. 2021,

unix.stackexchange.com/questions/668232/watchdog-bug-soft-lockup-cpu0-stuck-when-booting-arch-linux-on-minisforum. Accessed 26 Mar. 2025.



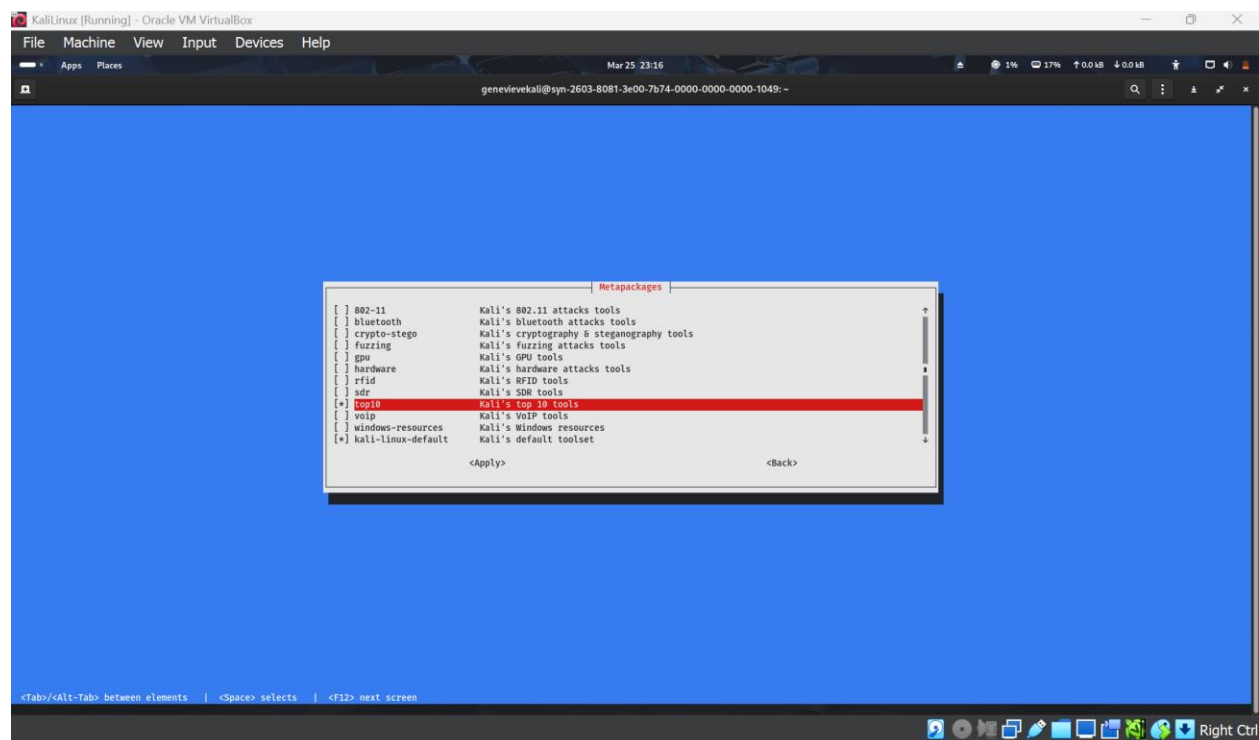


Figure 5: Above are the metapackages options.

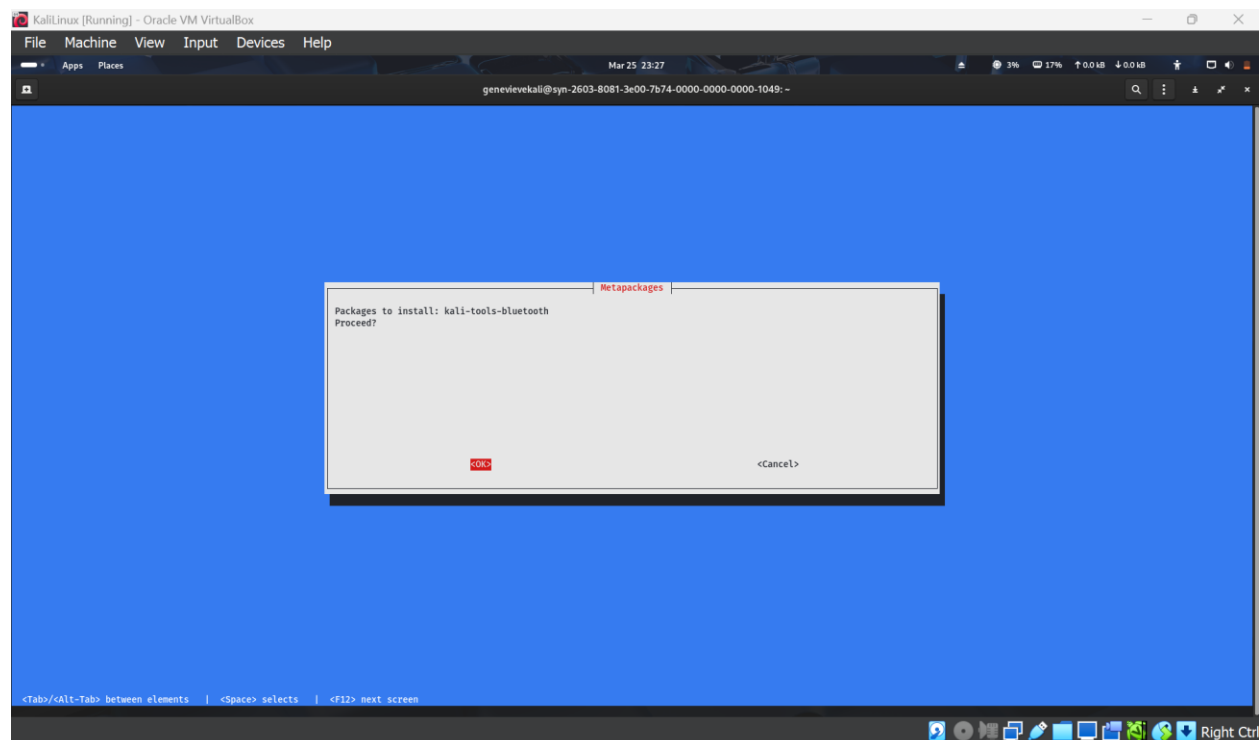


Figure 6: Above is a screenshot of the metapackage download options.

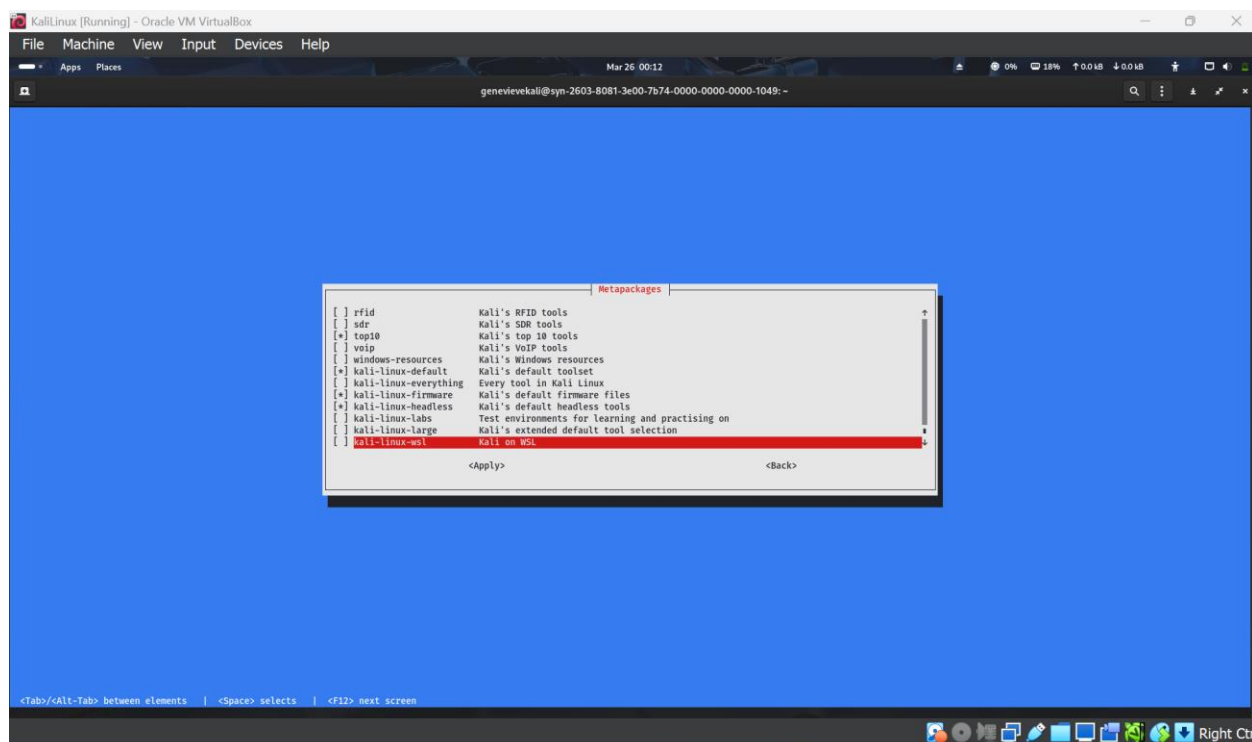


Figure 7: Above is a screenshot of all the metapackages that were previously downloaded.

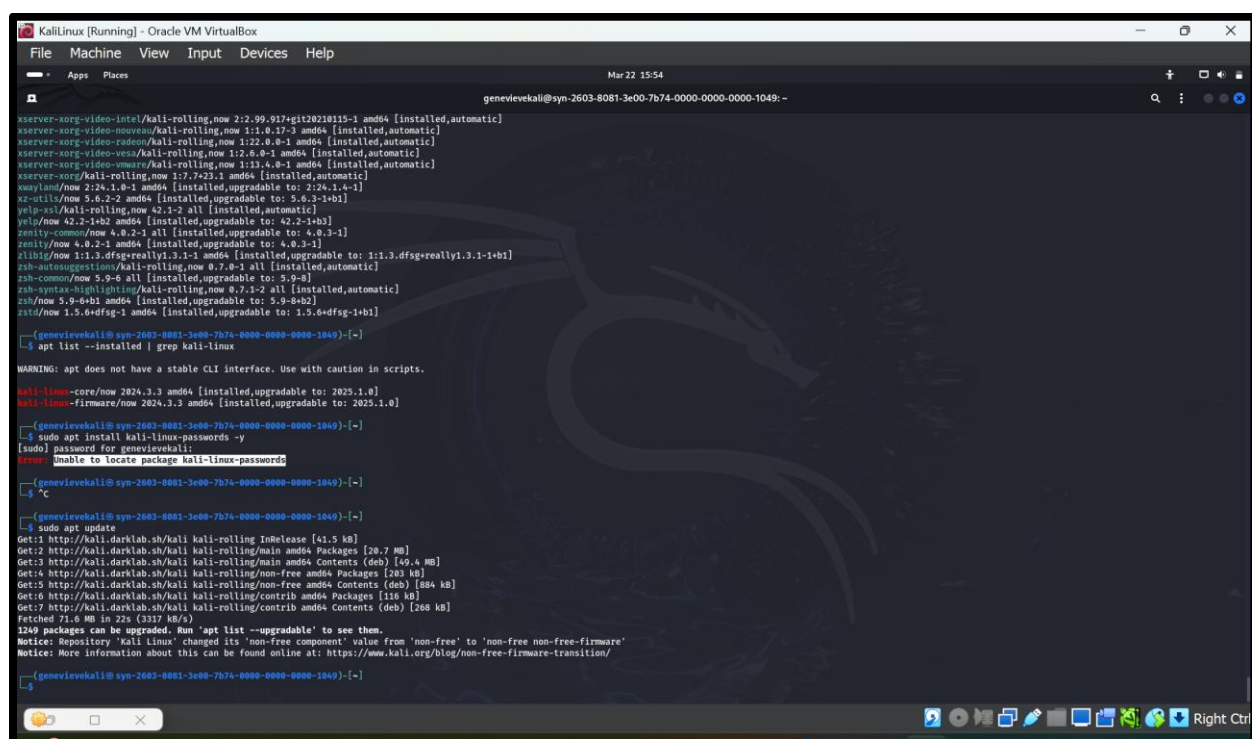


Figure 8: Entered “sudo apt install kali-linux-passwords -y” in order to download Hashcat, John the Ripper, and Hydra. It did not work, so I decided to update the package list by inputting “sudo apt update”.


```

KaliLinux [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Mar 22 17:00
genevievekall@syn-2603-8081-3e00-7b74-0000-0000-0000-1049: ~
Error: Unable to locate package kali-linux-passwords
(genevievekall@syn-2603-8081-3e00-7b74-0000-0000-0000-1049)~$
$ sudo apt update && sudo apt install johnny -y
Hit:1 http://http.kali.org/kali kali-rolling InRelease
962 packages can be upgraded. Run 'apt list --upgradable' to see them.
The following packages were automatically installed and are no longer required:
  chrome-gnome-shell
  cpp-13
  cpp-13-x86-64-linux-gnu
  gcc-13-base
  gir1.2-javascriptcoregtk-6.0
  gir1.2-webkit-6.0
  gnome-backgrounds
  gnome-shell-extension-auto-move-windows
  gnome-shell-extension-light-style
  gnome-shell-extension-native-window-placement
  gnome-shell-extension-screenshots-window-sizer
  gnome-shell-extension-windows-navigator
  gnome-shell-extension-workspace-indicator
  gnome-shell-extensions
  libdri3-dev
  libjavascriptcoregtk-6.0-1
  libperl5.38t64
  libpoppler134
  libproxy1-plugin-gsettings
  libproxy1-plugin-networkmanager
  libproxy1-plugin-webkit
  libpython3.11-minimal
  libpython3.11-stdlib
  libwebkit2gtk-6.0-4
  perl-modules-5.38
  python3.11
  python3.11-minimal
  xdg-user-dirs-gtk
Use 'sudo apt autoremove' to remove them.
Installing:
  johnny
Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 962
  Download size: 545 kB
  Space needed: 944 kB / 38.6 GB available
Get:1 http://http.kali.org/kali kali-rolling/main amd64 johnny amd64 2.2+git20160807-0kali2+b1 [545 kB]
Fetched 545 kB in 1s (517 kB/s)
Selecting previously unselected package johnny.
(Reading database ... 45341 files and directories currently installed.)
Preparing to unpack .../johnny_2.2+git20160807-0kali2+b1_amd64.deb ...
Unpacking johnny (2.2+git20160807-0kali2+b1) ...
Setting up johnny (2.2+git20160807-0kali2+b1) ...

```

Figure 11: Entered “sudo apt install kali-linux-default -y” then I decided to download Johnny as I did not see it on the VM. It successfully downloaded.

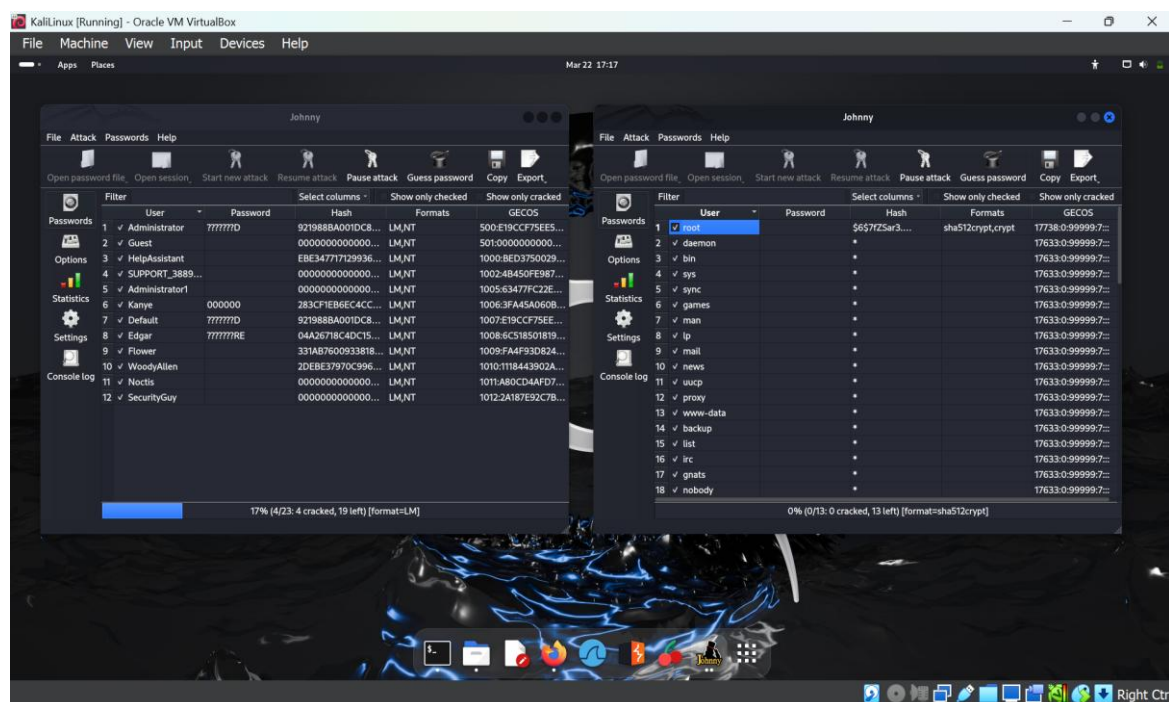


Figure 12: Running the two unzipped files on Johnny.

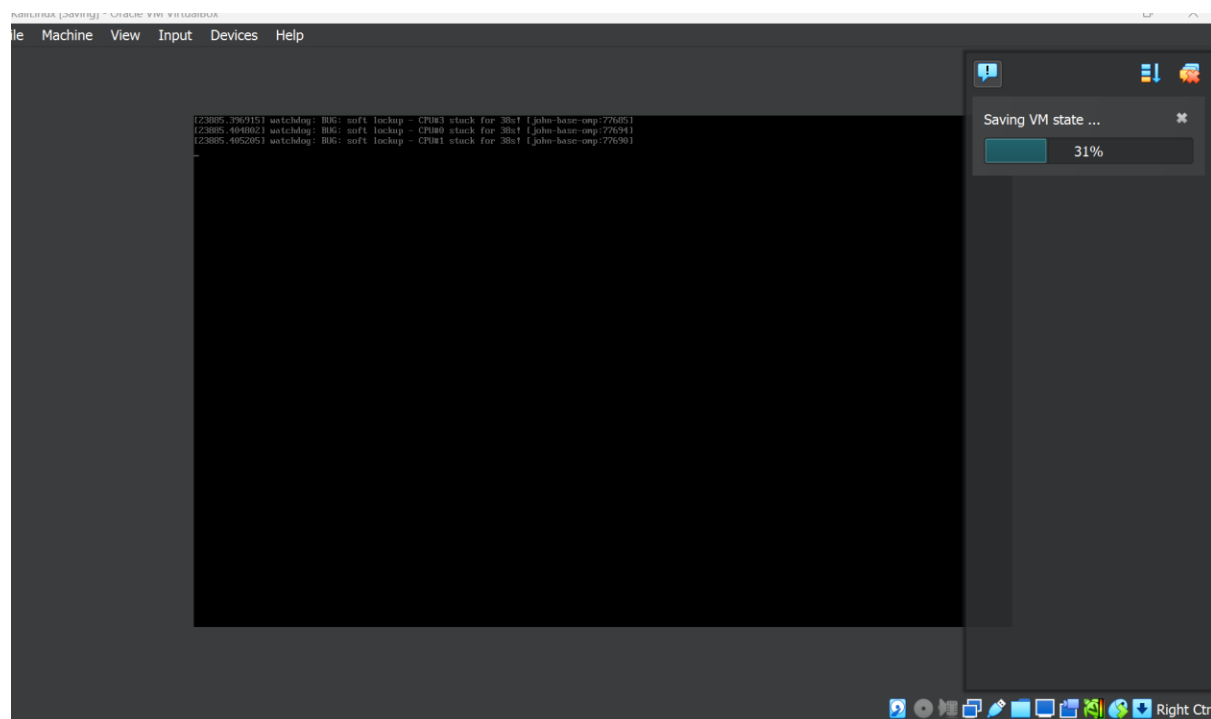


Figure 13: The VM ran for over 4 hours and then I received these error codes. I had to save the VM and restart. I changed the CPU to 6 in hopes this process would be a bit quicker.

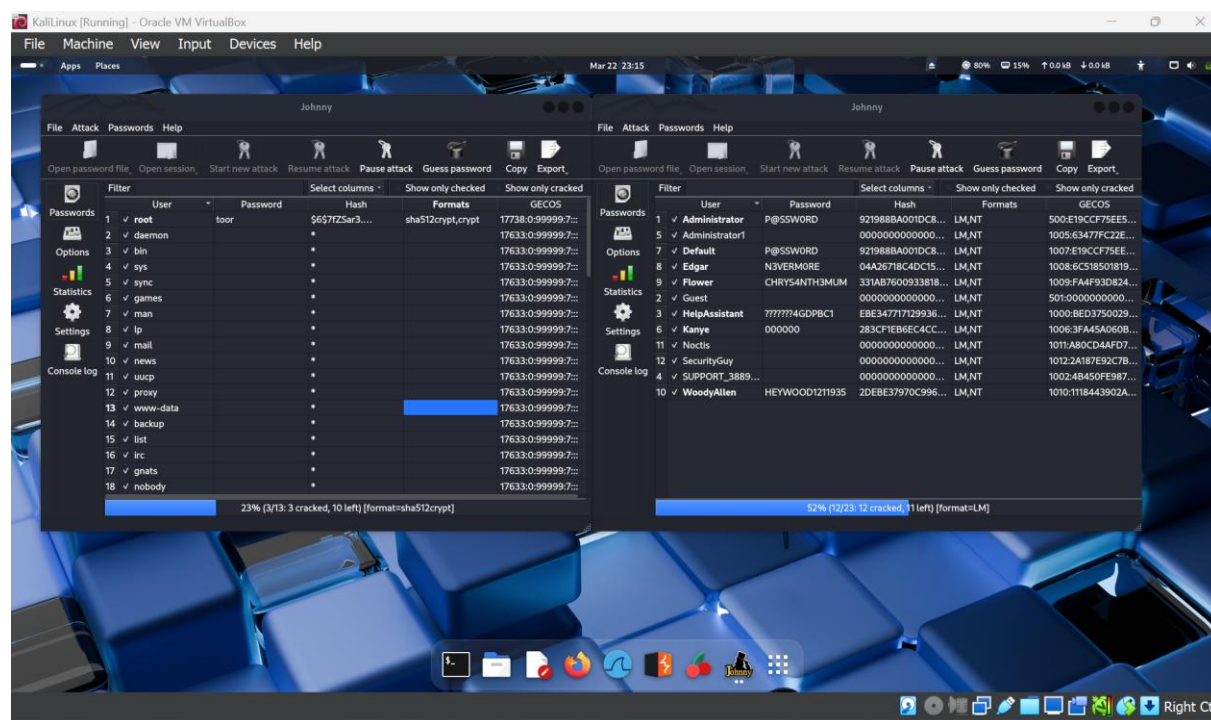


Figure 14: Above is the screenshot of how many passwords were cracked after waiting 6 hours. 3/13 for the shadow file and 12/23 were cracked for the SAM file.

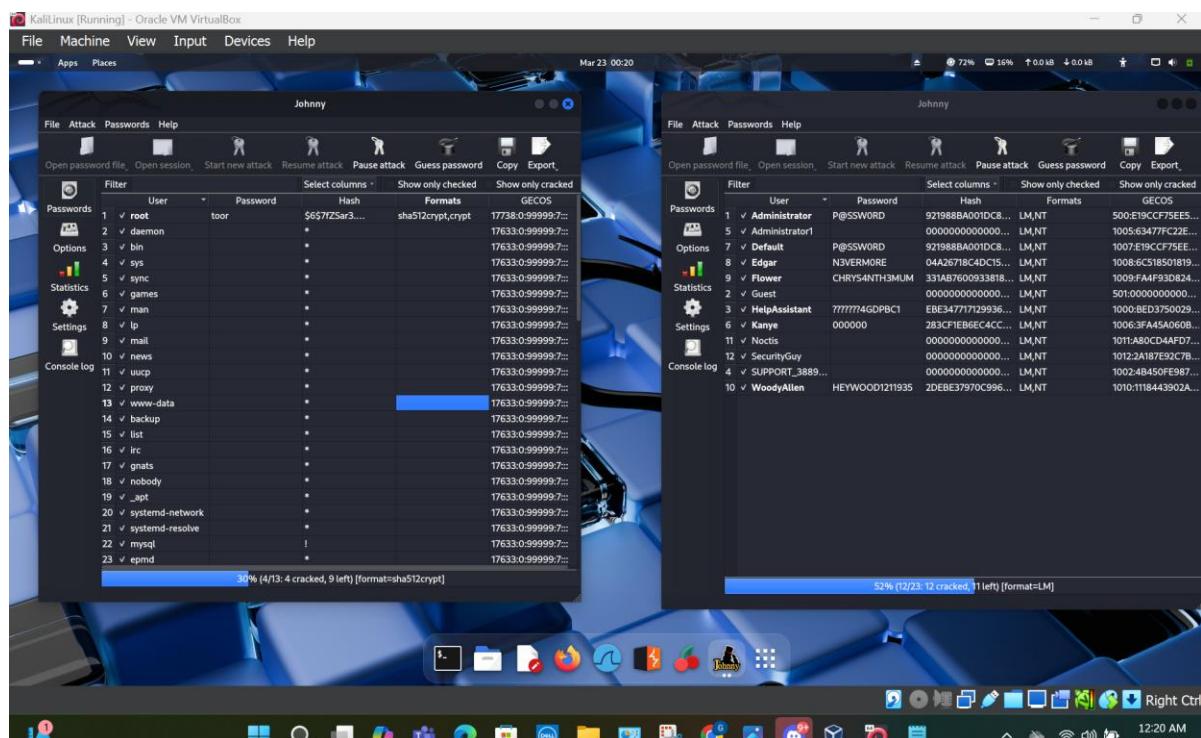


Figure 15: Above is a photo of 4/13 passwords cracked from the shadow file on the left (part 1). SAM's file of cracked passwords is on the right.

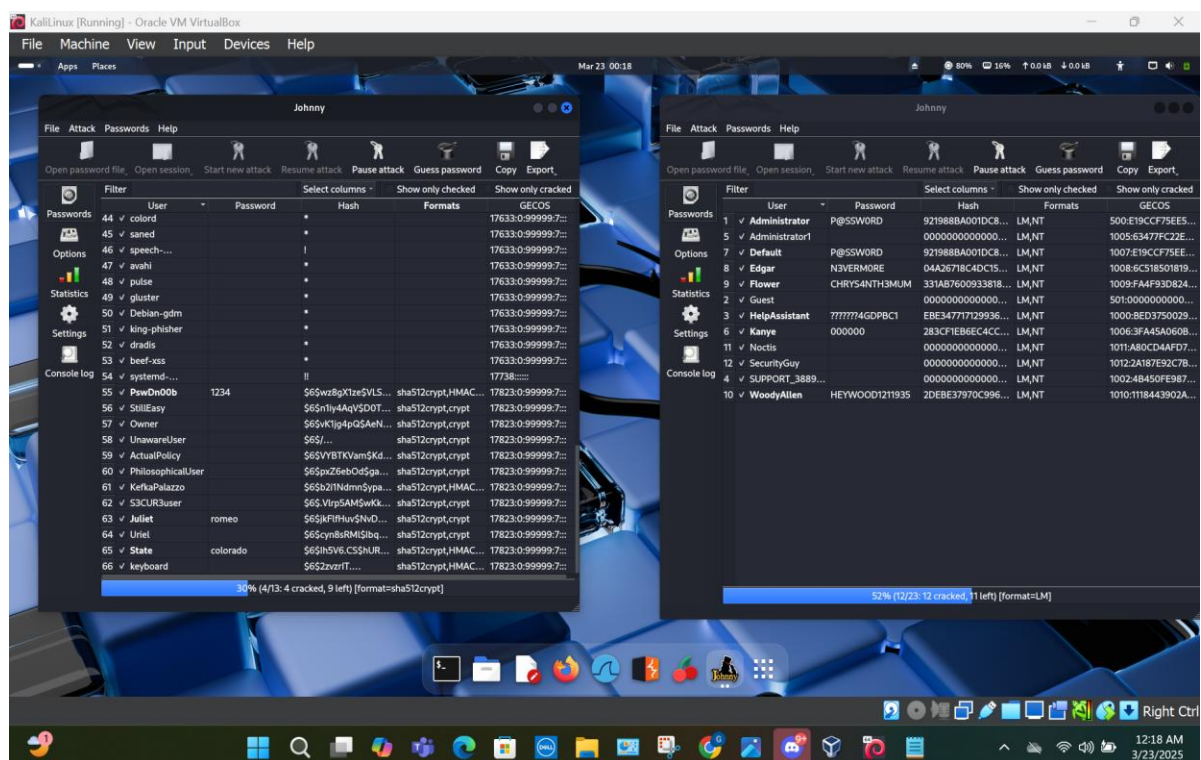


Figure 16: Above is a photo of the rest of the 4/13 passwords cracked of the shadow file on the left (part 2). SAM's file of cracked passwords is on the right.

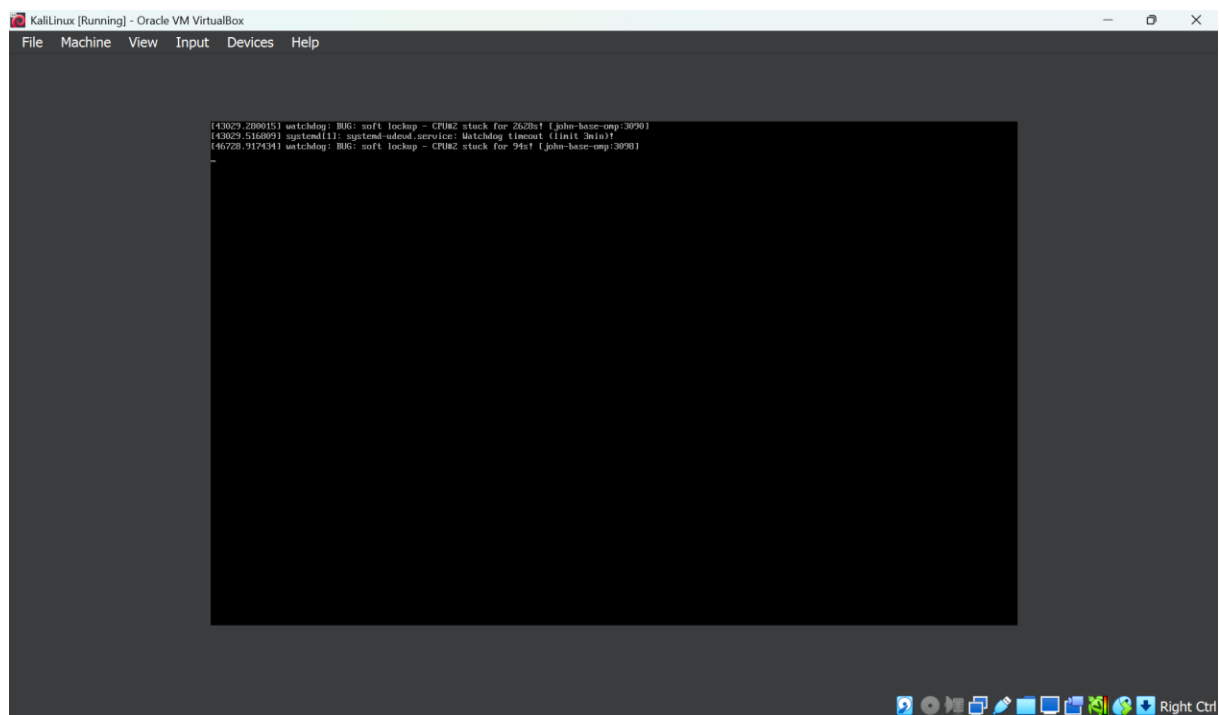


Figure 17: Error messages I received after leaving my machine running overnight.

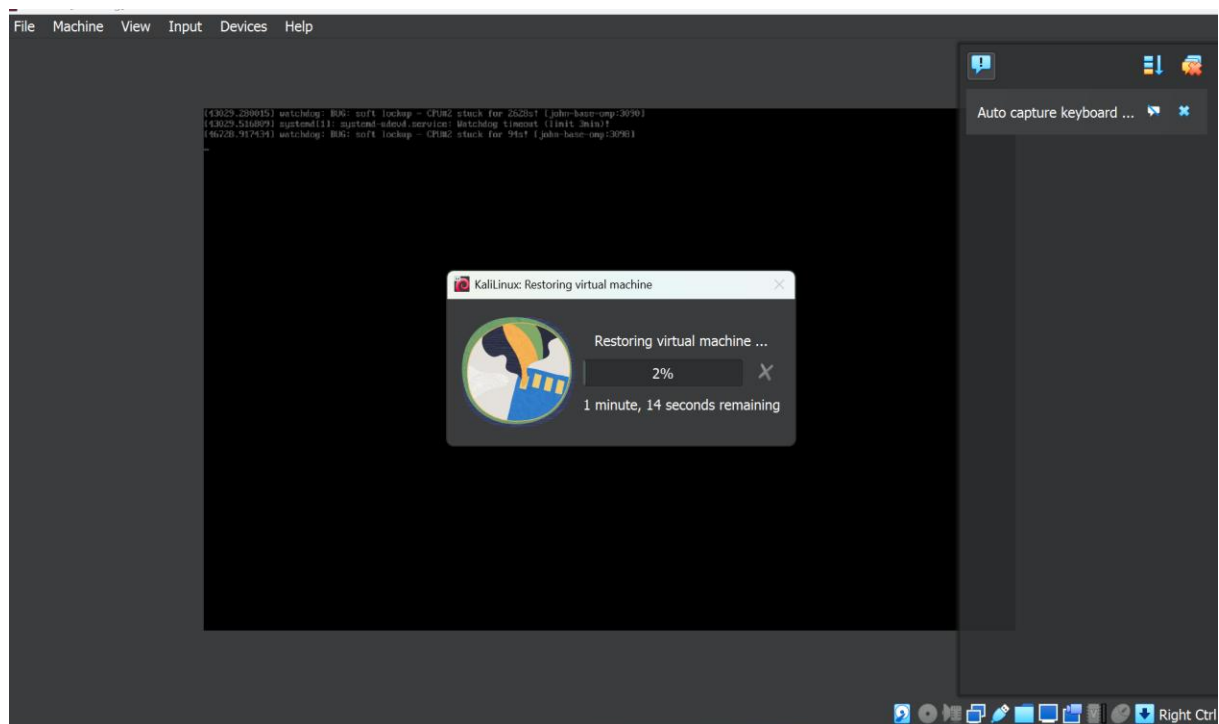


Figure 18: I saved and restarted the machine and received this error. I restarted just as I did before and was able to get back into the machine.

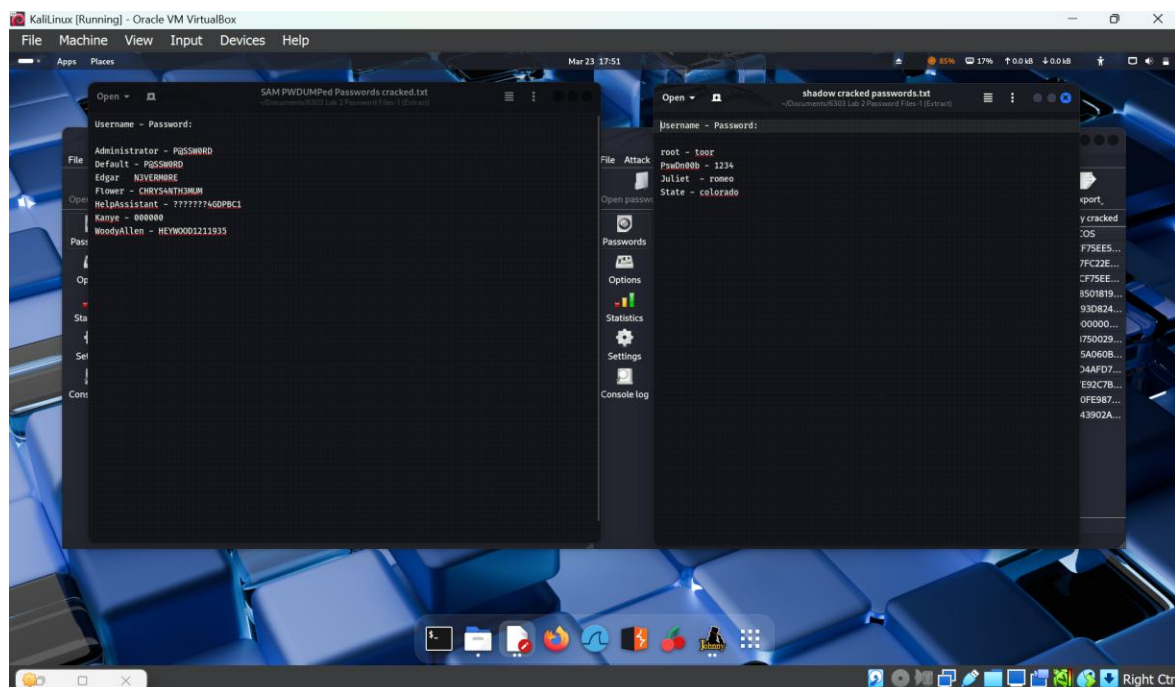


Figure 19: The files above include the cracked passwords from the two files from Johnny: this took about 24 hours total.

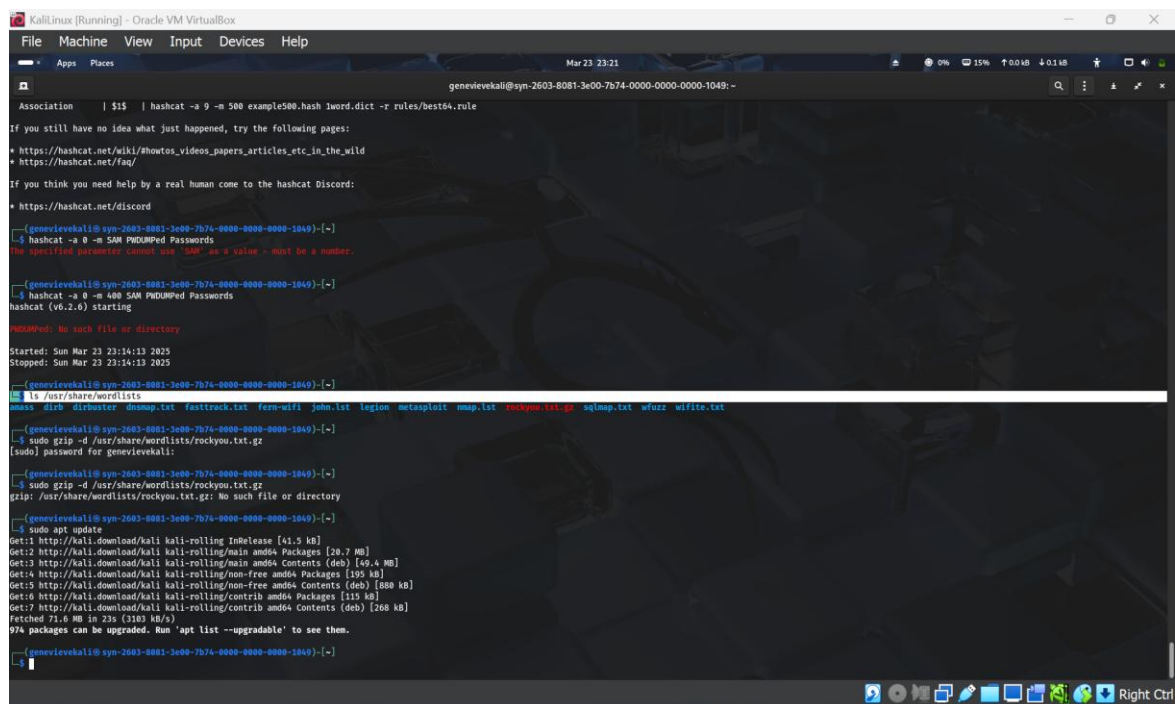
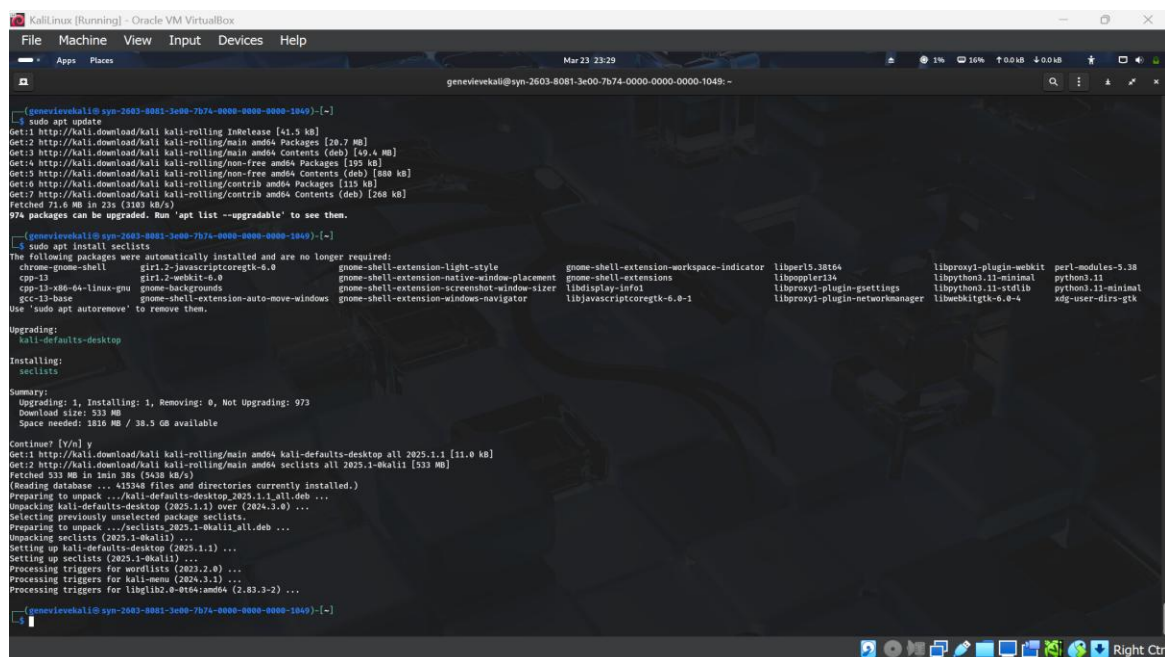


Figure 20: I inputted "ls /usr/share/wordlists" to see if I had "rockyou.txt.gz" - which I did. Unzipped the rockyou.txt.gz by inputting "sudo gzip -d /usr/share/wordlists/rockyou.txt.gz".



```

(genevievekali@syn-2603-8081-3e00-7b74-0000-0000-0000-1049)-[~]
$ sudo apt update
Get:1 http://kali.download/kali kali-rolling InRelease [41.5 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [26.9 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [49.4 MB]
Get:4 http://kali.download/kali kali-rolling/non-free amd64 Packages [193 kB]
Get:5 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [188 kB]
Get:6 http://kali.download/kali kali-rolling/contrib amd64 Packages [115 kB]
Get:7 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [268 kB]
Fetched 71.6 MB in 23s (3182 kB/s)
974 packages can be upgraded. Run 'apt list --upgradable' to see them.

(genevievekali@syn-2603-8081-3e00-7b74-0000-0000-0000-1049)-[~]
$ sudo apt install seclists
The following packages were automatically installed and are no longer required:
  chrome-gnome-shell glib2.0-javascriptcoregtk-0.0 gnome-shell-extension-light-style gnome-shell-extension-workspace-indicator libperl5.38t64 libproxy-plugin-webkit perl-modules-5.38
  cpp-13 glib2.0-webkit-0.0 gnome-shell-extension-native-window-placement gnome-shell-extensions libpoppler134 libpython3.11-minimal python3.11
  cpp-13-arm64-linux-gnu gnome-backgrounds gnome-shell-extension-screenshot-window-sizer libdisplay-info libproxy-plugin-gsettings libpython3.11-stdlib python3.11-minimal
  gcc-13-base gnome-shell-extension-auto-move-windows gnome-shell-extension-windows-navigator libjavascriptcoregtk-6.0-1 libproxy-plugin-netwotkmanager libwebkitgtk-0.0-4 xdg-user-dirs-gtk
Use 'sudo apt autoremove' to remove them.

Upgrading:
  kali-defaults-desktop

Installing:
  seclists

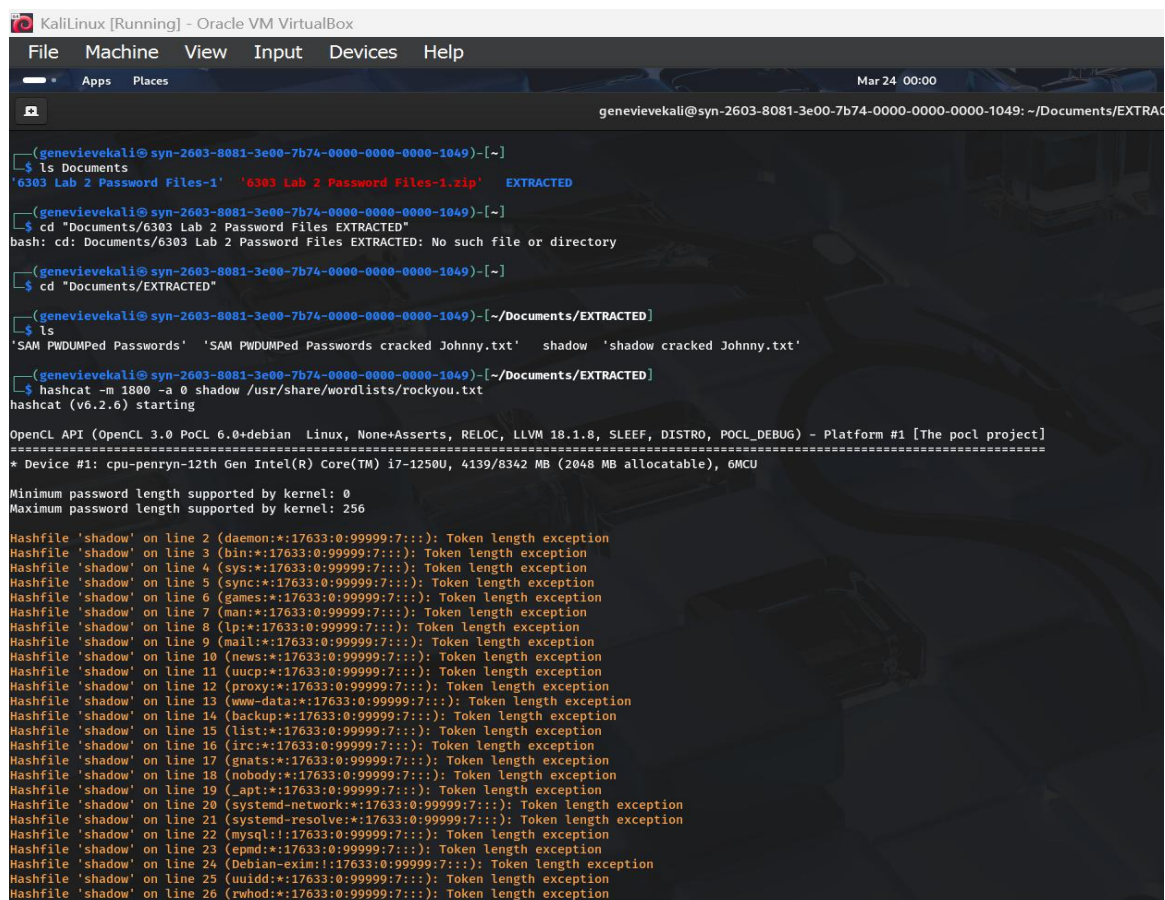
Summary:
  Upgrading: 1, Installing: 1, Removing: 0, Not Upgrading: 973
  Download size: 533 MB
  Space needed: 1010 MB / 38.5 GB available

Continue? [Y/n] y
Get:1 http://kali.download/kali kali-rolling/main amd64 kali-defaults-desktop all 2025.1.1 [11.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 seclists all 2025.1-0kali1 [533 MB]
Fetched 533 MB in 26s (20.2 MB/s)
(Reading database ... 415348 files and directories currently installed.)
Preparing to unpack .../kali-defaults-desktop-2025.1.1_all.deb ...
Unpacking kali-defaults-desktop (2025.1.1) over (2024.3.0) ...
Selecting previously unselected package seclists.
Preparing to unpack .../seclists-2025.1-0kali1_all.deb ...
Unpacking seclists (2025.1-0kali1) ...
Setting up kali-defaults-desktop (2025.1.1) ...
Setting up seclists (2025.1-0kali1) ...
Processing triggers for wordlists (2023.2.0) ...
Processing triggers for kali-menu (2024.3.1) ...
Processing triggers for libgl1:amd64 (2.0.0-2) ...

(genevievekali@syn-2603-8081-3e00-7b74-0000-0000-0000-1049)-[~]
$

```

Figure 21: Above is the photo of the SecList package download.



```

(genevievekali@syn-2603-8081-3e00-7b74-0000-0000-0000-1049)-[~]
$ ls Documents
'6303 Lab 2 Password Files-1' '6303 Lab 2 Password Files-1.zip' EXTRACTED

(genevievekali@syn-2603-8081-3e00-7b74-0000-0000-0000-1049)-[~]
$ cd "Documents/6303 Lab 2 Password Files EXTRACTED"
bash: cd: Documents/6303 Lab 2 Password Files EXTRACTED: No such file or directory

(genevievekali@syn-2603-8081-3e00-7b74-0000-0000-0000-1049)-[~]
$ cd "Documents/EXTRACTED"

(genevievekali@syn-2603-8081-3e00-7b74-0000-0000-0000-1049)-[~/Documents/EXTRACTED]
$ ls
'SAM PWDUMPed Passwords' 'SAM PWDUMPed Passwords cracked Johnny.txt' shadow 'shadow cracked Johnny.txt'

(genevievekali@syn-2603-8081-3e00-7b74-0000-0000-0000-1049)-[~/Documents/EXTRACTED]
$ hashcat -m 1800 -a 0 shadow /usr/share/wordlists/rockyou.txt
hashcat (v6.2.6) starting

OpenCL APT (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, LLVM 18.1.8, SLEEP, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]
=====
* Device #1: cpu-penryn-12th Gen Intel(R) Core(TM) i7-1250U, 4139/8342 MB (2048 MB allocatable), 6MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashfile 'shadow' on line 2 (daemon*:17633:0:99999:7::): Token length exception
Hashfile 'shadow' on line 3 (bin*:17633:0:99999:7::): Token length exception
Hashfile 'shadow' on line 4 (sys*:17633:0:99999:7::): Token length exception
Hashfile 'shadow' on line 5 (sync*:17633:0:99999:7::): Token length exception
Hashfile 'shadow' on line 6 (games*:17633:0:99999:7::): Token length exception
Hashfile 'shadow' on line 7 (man*:17633:0:99999:7::): Token length exception
Hashfile 'shadow' on line 8 (lp*:17633:0:99999:7::): Token length exception
Hashfile 'shadow' on line 9 (mail*:17633:0:99999:7::): Token length exception
Hashfile 'shadow' on line 10 (news*:17633:0:99999:7::): Token length exception
Hashfile 'shadow' on line 11 (uucp*:17633:0:99999:7::): Token length exception
Hashfile 'shadow' on line 12 (proxy*:17633:0:99999:7::): Token length exception
Hashfile 'shadow' on line 13 (www-data*:17633:0:99999:7::): Token length exception
Hashfile 'shadow' on line 14 (backup*:17633:0:99999:7::): Token length exception
Hashfile 'shadow' on line 15 (list*:17633:0:99999:7::): Token length exception
Hashfile 'shadow' on line 16 (irc*:17633:0:99999:7::): Token length exception
Hashfile 'shadow' on line 17 (gnats*:17633:0:99999:7::): Token length exception
Hashfile 'shadow' on line 18 (nobody*:17633:0:99999:7::): Token length exception
Hashfile 'shadow' on line 19 (_apt*:17633:0:99999:7::): Token length exception
Hashfile 'shadow' on line 20 (systemd-network*:17633:0:99999:7::): Token length exception
Hashfile 'shadow' on line 21 (systemd-resolve*:17633:0:99999:7::): Token length exception
Hashfile 'shadow' on line 22 (mysql*:17633:0:99999:7::): Token length exception
Hashfile 'shadow' on line 23 (cpanel*:17633:0:99999:7::): Token length exception
Hashfile 'shadow' on line 24 (Debian-exim:17633:0:99999:7::): Token length exception
Hashfile 'shadow' on line 25 (uiddd*:17633:0:99999:7::): Token length exception
Hashfile 'shadow' on line 26 (rwhod*:17633:0:99999:7::): Token length exception

```

Figure 22: Attempting to crack passwords from the shadow file using “hashcat -m 1800 -a 0 shadow /usr/share/wordlists/rockyou.txt”.

```

Kali Linux [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Mar 24 00:26
genevevekal@syn-2603-8081-3e00-7b74-0000-0000-0000-1049: ~/Documents/EXTRACTED

Progress: 2528896/186477005 (1.36%)
Rejected: 0/2528896 (0.00%)
Restore.Point: 194332/14343393 (1.36%)
Restore.Sub.#1: Salt:10 Amplifier:0-1 Iteration:1024-1536
Candidate.Engine: Device Generator
Candidates.#1: sws021 -> subsonica
Hardware.Mon.#1: Util: 70%

[s]status [p]pause [b]ypass [c]heckpoint [f]inish [q]uit => s
Session: hashcat
Status: Running
Hash.Mode: 1000 (sha512crypt $6$, SHA512 (unix))
Hash.Target: shadow
Time.Started: Sun Mar 23 23:49:25 2025 (33 mins, 20 secs)
Time.Estimated: Tue Mar 25 18:13:04 2025 (1 day, 9 hours)
Kernel.Feature: Pure Kernel
Guess.Base: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue: 1/1 (100.00%)
Speed.#1: 927 R/s (12.15ms) @ Accel:128 Loops:512 Thr:1 Vec:2
Recovered: 5/13 (38.46%) Digests (total), 5/13 (38.46%) Digests (new), 5/13 (38.46%) Salts
Progress: 3054464/186477005 (1.64%)
Rejected: 0/3054464 (0.00%)
Restore.Point: 234880/14343393 (1.64%)
Restore.Sub.#1: Salt:8 Amplifier:0-1 Iteration:3072-3504
Candidate.Engine: Device Generator
Candidates.#1: hass1ai -> sammy2
Hardware.Mon.#1: Util: 64%

[s]status [p]pause [b]ypass [c]heckpoint [f]inish [q]uit => s
Session: hashcat
Status: Running
Hash.Mode: 1000 (sha512crypt $6$, SHA512 (unix))
Hash.Target: shadow
Time.Started: Sun Mar 23 23:49:25 2025 (35 mins, 57 secs)
Time.Estimated: Tue Mar 25 18:34:08 2025 (1 day, 10 hours)
Kernel.Feature: Pure Kernel
Guess.Base: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue: 1/1 (100.00%)
Speed.#1: 917 R/s (10.99ms) @ Accel:128 Loops:512 Thr:1 Vec:2
Recovered: 5/13 (38.46%) Digests (total), 5/13 (38.46%) Digests (new), 5/13 (38.46%) Salts
Progress: 3283072/186477005 (1.76%)
Rejected: 0/3283072 (0.00%)
Restore.Point: 252844/14343393 (1.76%)
Restore.Sub.#1: Salt:1 Amplifier:0-1 Iteration:4608-5000
Candidate.Engine: Device Generator
Candidates.#1: 41190 -> 310303
Hardware.Mon.#1: Util: 64%

[s]status [p]pause [b]ypass [c]heckpoint [f]inish [q]uit =>

```

Figure 23: Above is the progress of Hashcat for the shadow file before realizing there was an error.

```

Kali Linux [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Mar 24 01:37
shadow hashes only.txt
~/.ssh/EXTRACTED

shadow hashes only.txt

$6$7fZsar3.$mUE2gD5P7EY3qP481AQIU41f45DCfux6515Mh81DokLPLGUiDa5F52aFUEPcixVdv1Rb0t1Egxm8KwUFDVR
$6$7fZsar3.$mUE2gD5P7EY3qP481AQIU41f45DCfux6515Mh81DokLPLGUiDa5F52aFUEPcixVdv1Rb0t1Egxm8KwUFDVR
$6$w28gK1ze$VLS-/5bq.fHLSUX05vAr05AcBz0q0W0a121xRzAuAoLVIM7K8Dp0b0jdy/Z2RWH5ureL3e38q28d1N0
$6$11yAqy$D0TYua3bsFEIYguLSKTH5jgXjpm/Lq4VRIJMO3VY1bKa5v2Fm09.1a6ty.RfkdZ7H1e2Ha2400n.bt3J80
$6$vk1jg4p0AeHeZet2f1tb7fMeJ0MaliwCveb30Vb0n61C/cgDhV0v0ndKxTv4808rc0V3/16uH7J0y2CX0vA1h9q98Lq11
$6$/CTSyb4d$0oVpD248RLs2m6y3MSDem73Cvfo8zob70B0J35Ve0pTL1svZ8SUIu1LaYqDxRH2WIEY3ryBZg2/r0d0.
$6$YVBTkVam$KdPtmsyc4-57zZzXWEX4j20cCN0p7KyaJm0TfShC2Gm0V0B1x5YDze7N11Nny3rqlEZNmTISg2mIG60/
$6$pxZ6eb0d$gaZnx7zfJgkPFXL/c/Dz5TmBfz/0M2BxZyR8pVXK4H05G1Xmz0ZDmKfVfIDP.zr3JBSAW.k1010W1DqB/
$6$bz11ndm5ypaGc.06z1Y0T7Hikm.ra9j1UWhu0R3pY1E376/gp0Jk9kmULdA5UGS6we0021Nxy9A0cycgvFN3M04Z/
$6$.Vlrp5AM5wKc6UEI3tEqEQ/kokSPK7pq9wTA8RmHycwIY817oFpXSD1ruyAg1FFS0e.F191PznLTt2wN4ZPhoz1p0
$6$JkFl1Huv$Nv0RwK3D6du7Ej03k1ChJ.UBfGV3Pzf1RefTb10EscnE1.qADGwe4CPMwJk6cHneI96A/VJ03Vv0Q011
$6$cyn8SRML$1bg0bb0kxw0uZwFCD6zTPzR80RYx86yeA91cPbrMjgM2eJukXorq2LLw0MH6gB2Z1PKAfc.bwJf3cq8P0
$6$1h5V6.CS$H0RkjkG1AA6153zbAry40pzu18Y3pBUZJk25jx.F1mPmV/.pG6LwJfYt.K.R7ebv.OP8ra3m/oc3BLLPx0
$6$2zvzrIT.5V8wFyfr-7/cZVcoP0N191f88g0NLL1j23NjU57J04eE3IwIaPme0pEntdSGS5t0GvC0tb1Zs6ws/HV10

```

Figure 24: I received an error code for using the original file for shadow, so then I created a new file called “shadow hashes only.txt”. I removed the usernames and any extra words/symbols that were not part of the hash.

```

Kali Linux [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Mar 24 01:39
genevievekall@sym-2603-8081-3e00-7b74-0000-0000-1049:~/Documents/EXTRACTED

Started: Mon Mar 24 01:16:11 2025
Stopped: Mon Mar 24 01:16:11 2025

genevievekall@sym-2603-8081-3e00-7b74-0000-0000-1049:~/Documents/EXTRACTED$
hashcat -m 1800 -a 0 "shadow hashes only.txt" /usr/share/wordlists/rockyou.txt
hashcat (v6.2.6) starting

OpenCL API (OpenCL 3.0 PoCL 6.0 Debian Linux, None+Asserts, RELOC, LLVM 16.1.8, SLEEP, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]
=====
* Device #1: cpu-penryn-12th Gen Intel(R) Core(TM) i7-12580U, 4139/8342 MB (2048 MB allocatable), 6MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashfile 'shadow hashes only.txt' on line 1 ($6$7f2...afUEPcIxdvIBBt1iGomKNUFDVW): Token length exception
Hashfile 'shadow hashes only.txt' on line 3 ($6$7f2...52aFUEPcJxdvJHb1iGomKNUFDVW): Token length exception

* Token length exception: 2/14 hashes
This error happens if the wrong hash type is specified, if the hashes are
malformed, or if input is otherwise not as expected (for example, if the
--username option is used but no username is present)

Hashes: 12 digests; 12 unique digests, 12 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Uses-64-Bit

ATTENTION! Pure (unoptimized) backend kernels selected.
Pure kernels can crack longer passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Temperature abort trigger set to 90c

INFO: Removed 5 hashes found as potfile entries.

Host memory required for this attack: 0 MB

Dictionary cache hit:
* Filename..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344385
* Bytes.....: 139921587
* Keyspace..: 14344385

Cracking performance lower than expected?
* Append -O to the commandline.
This lowers the maximum supported password/salt length (usually down to 32).

```

Figure 25: I then re-ran Hashcat “hashcat -m 1800 -a 0 “shadow hashes only.txt” /usr/share/wordlists/rockyou.txt”.

```

Kali Linux [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Mar 24 01:57
genevievekall@sym-2603-8081-3e00-7b74-0000-0000-1049:~/Documents/EXTRACTED

Progress.....: 2596512/172132620 (1.51%)
Rejected.....: 0/2596512 (0.00%)
Restore.Point...: 216352/14344385 (1.51%)
Restore.Sub.#1...: Salt:9 Amplifier:0-1 Iteration:4096-5000
Candidate.Engine: Device Generator
Candidates.#1....: metsof -> meowchie
Hardware.Mon.#1..: Util: 55%

[s]status [p]ause [b]ypass [c]heckpoint [f]inish [q]uit => s

Session.....: hashcat
Status.....: Running
Hash.Mode.....: 1800 (sha512crypt $6$, SHA512 (unix))
Hash.Target....: shadow hashes only.txt
Time.Started...: Mon Mar 24 01:17:39 2025 (37 mins, 8 secs)
Time.Estimated...: Tue Mar 25 08:29:47 2025 (1 day, 6 hours)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 807 M/s (5.83ms) @ Accel:32 Loops:1824 Thr1 Vec:2
Recovered.....: 5/12 (41.67%) Digests (total), 0/12 (0.00%) Digests (new), 5/12 (41.67%) Salts
Progress.....: 2858784/172132620 (1.66%)
Rejected.....: 0/2858784 (0.00%)
Restore.Point...: 238508/14344385 (1.66%)
Restore.Sub.#1...: Salt:10 Amplifier:0-1 Iteration:4096-5000
Candidate.Engine: Device Generator
Candidates.#1....: mommyboy -> mommyboy
Hardware.Mon.#1..: Util: 48%

[s]status [p]ause [b]ypass [c]heckpoint [f]inish [q]uit => s

Session.....: hashcat
Status.....: Running
Hash.Mode.....: 1800 (sha512crypt $6$, SHA512 (unix))
Hash.Target....: shadow hashes only.txt
Time.Started...: Mon Mar 24 01:17:39 2025 (39 mins, 11 secs)
Time.Estimated...: Tue Mar 25 06:23:37 2025 (1 day, 4 hours)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 963 M/s (4.47ms) @ Accel:32 Loops:1824 Thr1 Vec:2
Recovered.....: 5/12 (41.67%) Digests (total), 0/12 (0.00%) Digests (new), 5/12 (41.67%) Salts
Progress.....: 3855136/172132620 (1.77%)
Rejected.....: 0/3855136 (0.00%)
Restore.Point...: 254592/14344385 (1.77%)
Restore.Sub.#1...: Salt:11 Amplifier:0-1 Iteration:4096-5000
Candidate.Engine: Device Generator
Candidates.#1....: 15851982 -> 14723
Hardware.Mon.#1..: Util: 60%

[s]status [p]ause [b]ypass [c]heckpoint [f]inish [q]uit =>

```

Figure 26: I then was able to crack 4 passwords in the shadow file using Hashcat.

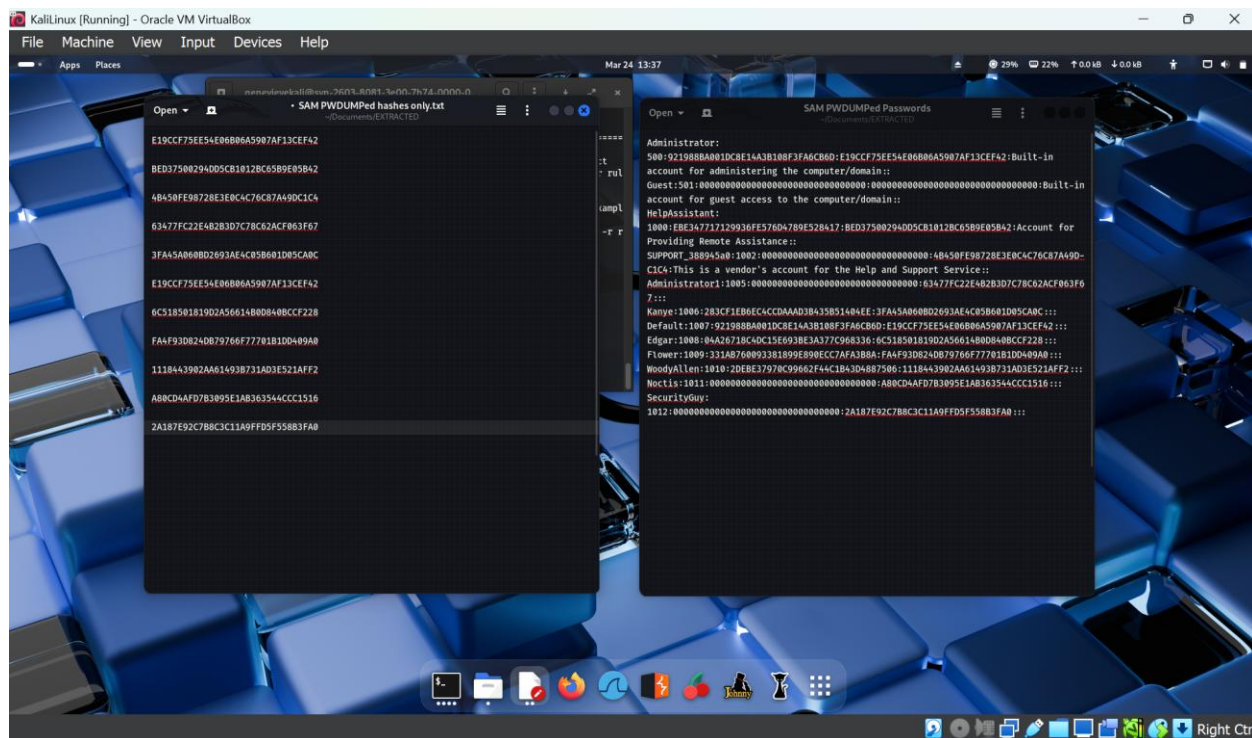


Figure 29. Before (right) and after (left) of removing everything but the hashes from the original SAM PWDUMPed Passwords file.

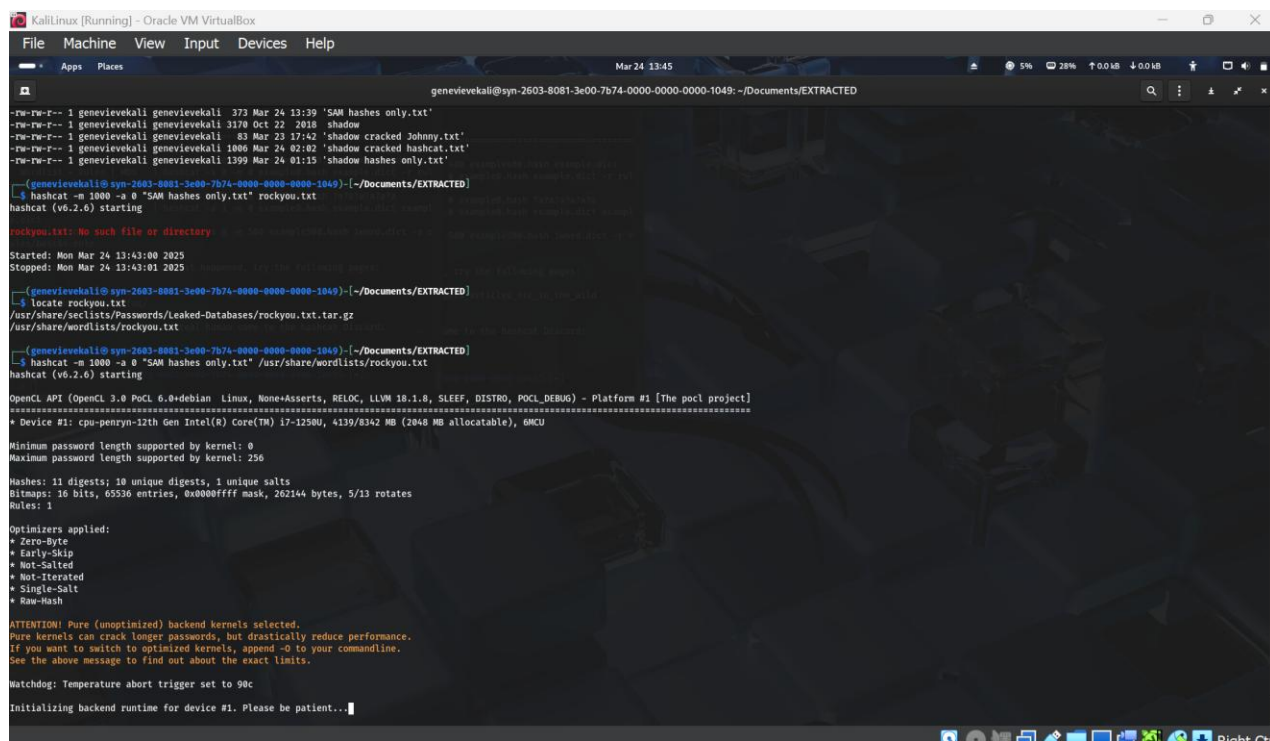


Figure 30: I had a variety of error codes but I was finally able to ensure that “rockyou.txt” was in the correct directory. I then entered “hashcat -m 1000 -a 0 “SAM hashes only.txt” /usr/share/wordlists/rockyou.txt” to initiate Hashcat for the SAM file.


```

KaliLinux [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Mar 24 13:48
genevevekali@syn-2603-8081-3e00-7b74-0000-0000-1049: ~/Documents/EXTRACTED

* Create more work items to make use of your parallelization power:
https://hashcat.net/faq/morework

[s]tatus [p]ause [b]ypass [c]heckpoint [f]inish [q]uit => s

Session.....: hashcat
Status.....: Running
Hash.Mode.....: 1000 (NTLM)
Hash.Target.....: SAM hashes only.txt
Time.Started....: Mon Mar 24 13:48:07 2025 (11 secs)
Time.Estimated...: Mon Mar 24 13:48:18 2025 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 1177.0 kH/s (1.03ms) @ Accel:1024 Loops:1 Thr:1 Vec:4
Recovered.....: 2/10 (20.00%) Digests (total), 0/10 (0.00%) Digests (new)
Progress.....: 13461504/14344385 (93.85%)
Rejected.....: 0/13461504 (0.00%)
Restore.Point....: 13461504/14344385 (93.85%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#1....: 101503a -> 100preant
Hardware.Mon.#1...: Util: 48%

Approaching final keypace - workload adjusted.

Session.....: hashcat
Status.....: Exhausted
Hash.Mode.....: 1000 (NTLM)
Hash.Target.....: SAM hashes only.txt
Time.Started....: Mon Mar 24 13:48:07 2025 (13 secs)
Time.Estimated...: Mon Mar 24 13:48:20 2025 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 1077.7 kH/s (1.01ms) @ Accel:1024 Loops:1 Thr:1 Vec:4
Recovered.....: 2/10 (20.00%) Digests (total), 0/10 (0.00%) Digests (new)
Progress.....: 14344385/14344385 (100.00%)
Rejected.....: 0/14344385 (0.00%)
Restore.Point....: 14344385/14344385 (100.00%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#1....: $HEX[216361726f6c796e] -> $HEX[042a0337c2a156616d6f732103]
Hardware.Mon.#1...: Util: 85%

Started: Mon Mar 24 13:48:06 2025
Stopped: Mon Mar 24 13:48:22 2025

(genevevekali@syn-2603-8081-3e00-7b74-0000-0000-1049)~[~/Documents/EXTRACTED]
$

```

Figure 31: Cracked 2 passwords.

```

KaliLinux [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Mar 24 13:53
genevevekali@syn-2603-8081-3e00-7b74-0000-0000-1049: ~/Documents/EXTRACTED

Session.....: hashcat
Status.....: Running
Hash.Mode.....: 1000 (NTLM)
Hash.Target.....: SAM hashes only.txt
Time.Started....: Mon Mar 24 13:48:07 2025 (11 secs)
Time.Estimated...: Mon Mar 24 13:48:18 2025 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 1177.0 kH/s (1.03ms) @ Accel:1024 Loops:1 Thr:1 Vec:4
Recovered.....: 2/10 (20.00%) Digests (total), 0/10 (0.00%) Digests (new)
Progress.....: 13461504/14344385 (93.85%)
Rejected.....: 0/13461504 (0.00%)
Restore.Point....: 13461504/14344385 (93.85%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#1....: 101503a -> 100preant
Hardware.Mon.#1...: Util: 48%

Approaching final keypace - workload adjusted.

Session.....: hashcat
Status.....: Exhausted
Hash.Mode.....: 1000 (NTLM)
Hash.Target.....: SAM hashes only.txt
Time.Started....: Mon Mar 24 13:48:07 2025 (13 secs)
Time.Estimated...: Mon Mar 24 13:48:20 2025 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 1077.7 kH/s (1.01ms) @ Accel:1024 Loops:1 Thr:1 Vec:4
Recovered.....: 2/10 (20.00%) Digests (total), 0/10 (0.00%) Digests (new)
Progress.....: 14344385/14344385 (100.00%)
Rejected.....: 0/14344385 (0.00%)
Restore.Point....: 14344385/14344385 (100.00%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidate.Engine.: Device Generator
Candidates.#1....: $HEX[216361726f6c796e] -> $HEX[042a0337c2a156616d6f732103]
Hardware.Mon.#1...: Util: 85%

Started: Mon Mar 24 13:48:06 2025
Stopped: Mon Mar 24 13:48:22 2025

(genevevekali@syn-2603-8081-3e00-7b74-0000-0000-1049)~[~/Documents/EXTRACTED]
$ hashcat -m 1000 -a 0 "SAM hashes only.txt" /usr/share/wordlists/rockyou.txt --show
e19ccf75ee54e8b08a5987af13cfa2:Pqssword
3fa5a00bbd2932e4c50a01d05cabc:000000

(genevevekali@syn-2603-8081-3e00-7b74-0000-0000-1049)~[~/Documents/EXTRACTED]
$

```

Figure 32: Above is the cracked passwords list, I inputted "hashcat -m 1000 -a 0 "SAM hashes only.txt" /usr/share/wordlists/rockyou.txt --show".

```
File Machine View Input Devices Help
Apps Places
Mar 24 17:55
geneveevkali@syn-2003-8081-3e00-7b74-0000-0000-0000-1049: -
--proxy_port: Proxy port, default 8080.
--proxy_username: Username for proxy, if required.
--proxy_password: Password for proxy, if required.

Headers
--header, -H: In format name:value - can pass multiple.

<url>: The site to spider.

(geneveevkali@syn-2003-8081-3e00-7b74-0000-0000-0000-1049):~$
$ curl -d 2 -m 5 -w custom.txt https://education.utaa.edu/faculty/
ent.62.1 (More Fixes) Robin Wood (robinwdigi.ninja) (https://digi.ninja/)
(geneveevkali@syn-2003-8081-3e00-7b74-0000-0000-0000-1049):~$
$ cat custom.txt
Education
Google
Manager
Header
Antonio
University
Texas
Development
Research
Faculty
Studies
College
Human
Request
Programs
Apply
Visit
noscript
headrunner
C000
Counseling
Student
Educational
education
Information
Resources
Section
Emergency
Footer
Notice
Scripts
Leadership
Teaching
Policy
Bilingual
Professor
Community
```

Figure 34: The figure above is in relation to CEWL. I inputted the code “cewl -d 2 -m 5 -w custom.txt <https://education.utsa.edu/faculty/>”. Targeting: <https://education.utsa.edu/faculty/>. “-w custom.txt” saved the words that were scraped into custom.txt, as request in the instructions. “-m 5” extracts the words that are 5 or more characters. Finally, “-d 2” requests to only follows links that are from the main page all the way to the subpages. I also entered “cat” and it showed that there are a large amount of words that were extracted, but no names.

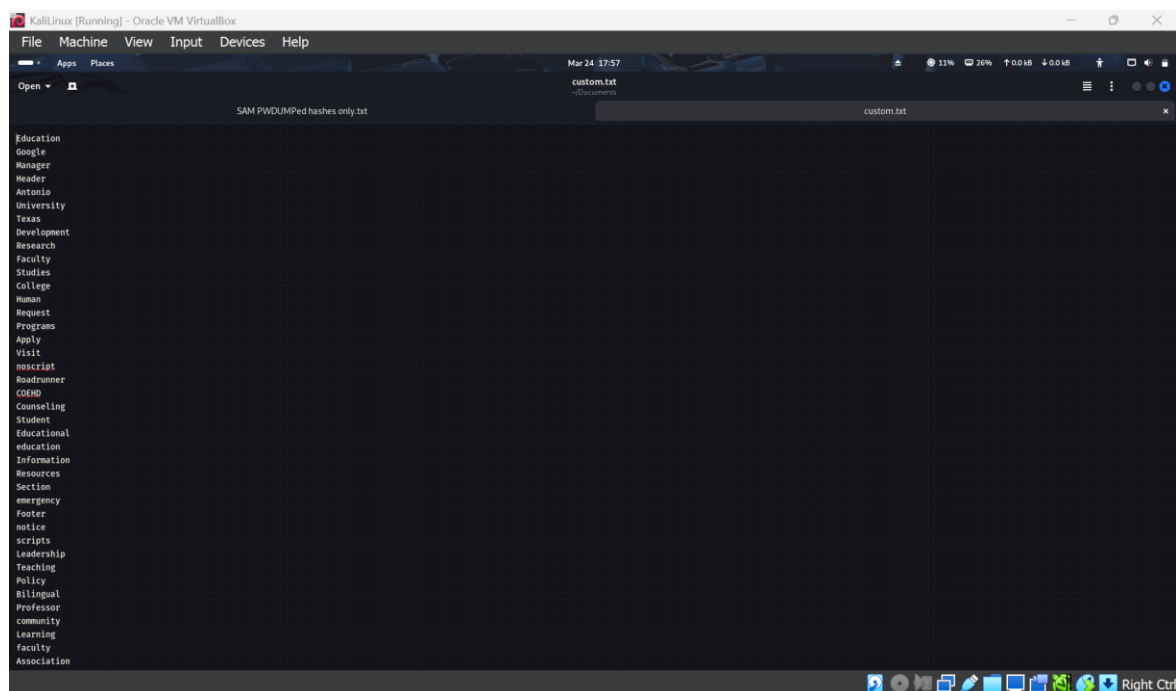


Figure 35: Above is a closer look at the file I created called custom.txt. Which I deleted to use later in Part 3.

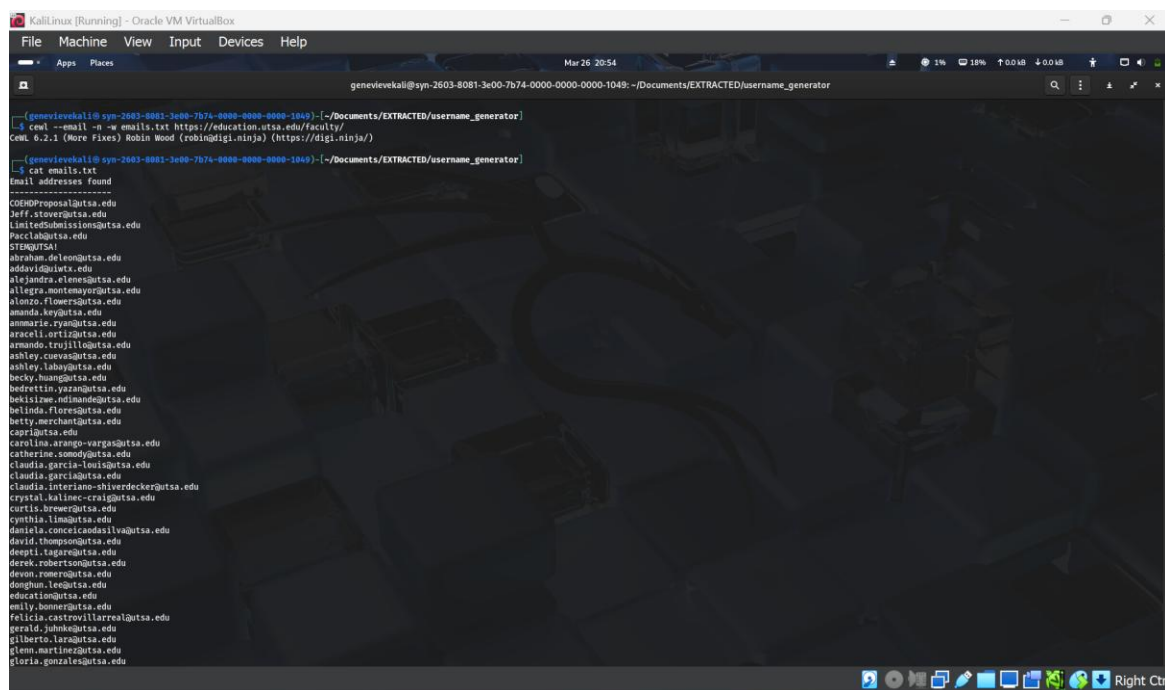
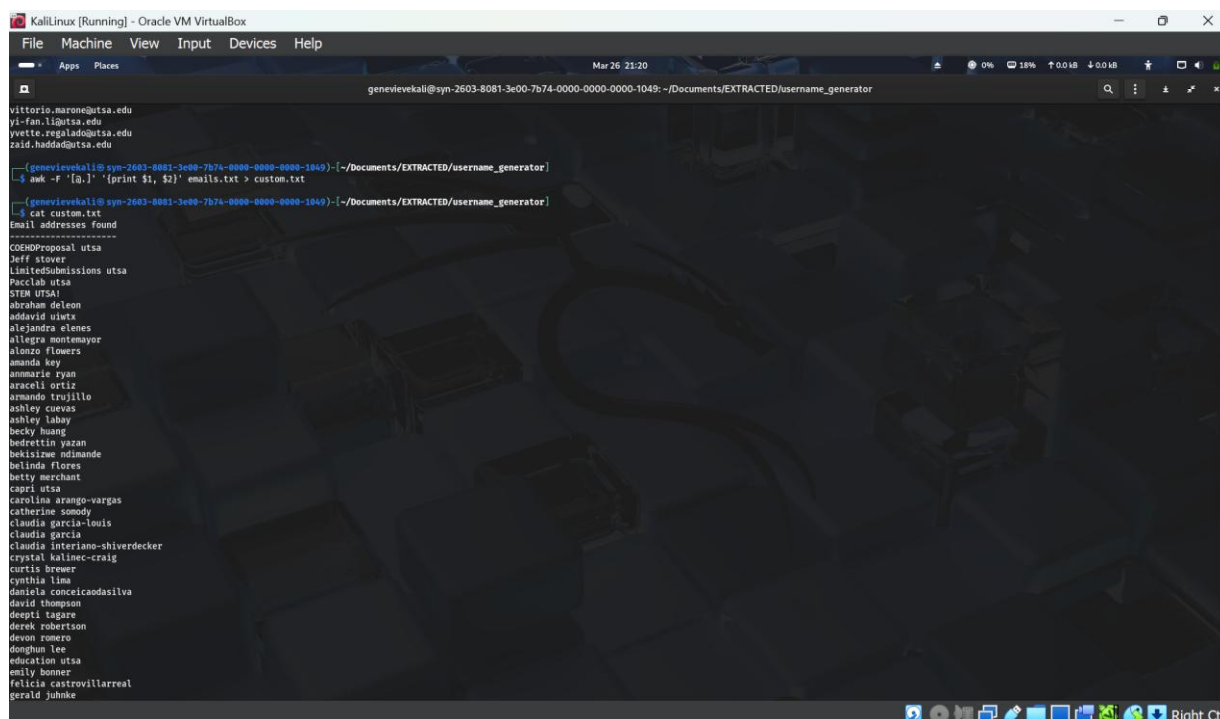


Figure 36: The figure above shows the emails that were added from the website (into the emails.txt).

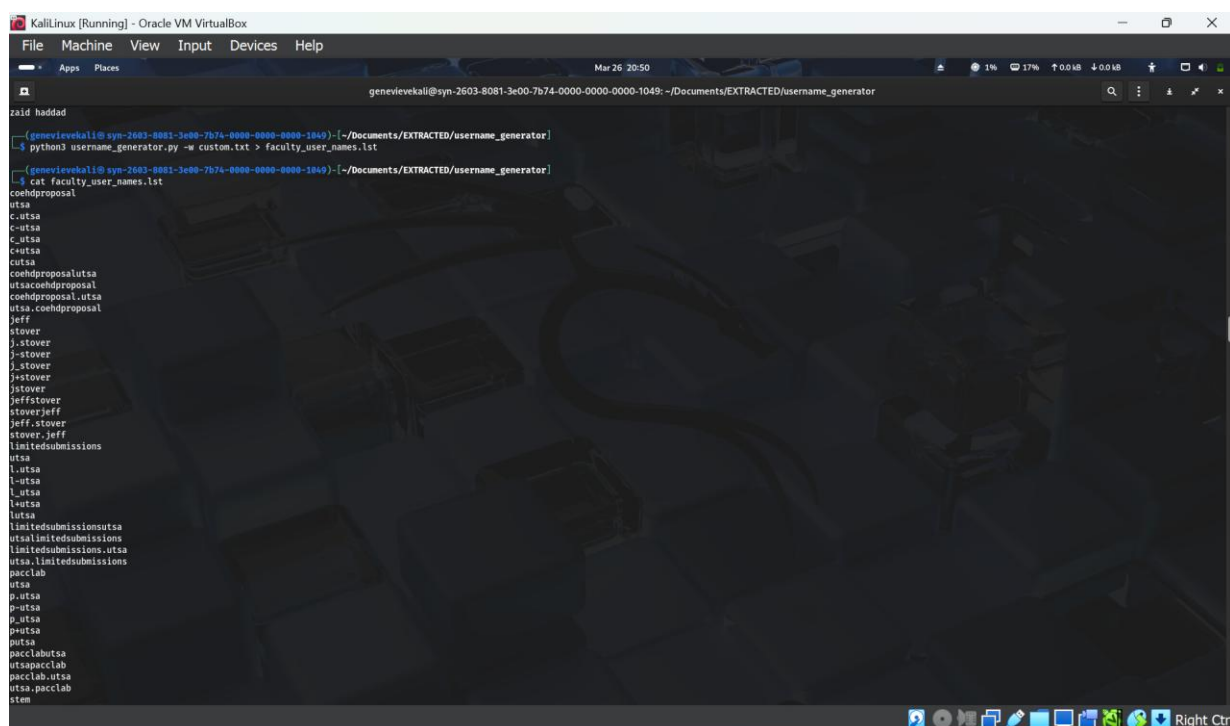


```

Kali Linux [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Mar 26 21:20
genevevekali@syn-2603-8081-3e00-7b74-0000-0000-0000-1049: ~/Documents/EXTRACTED/username_generator
vittorio.marone@utsa.edu
yi-fan.li@utsa.edu
yvette.regalado@utsa.edu
zaid.haddad@utsa.edu
genevevekali@syn-2603-8081-3e00-7b74-0000-0000-1049:~/Documents/EXTRACTED/username_generator
$ awk -F '@.' '{print $1, $2}' emails.txt > custom.txt
genevevekali@syn-2603-8081-3e00-7b74-0000-0000-1049:~/Documents/EXTRACTED/username_generator
$ cat custom.txt
Email addresses found
-----
COENHProposal@utsa
Jeff@stover
LimitedSubmissions@utsa
Pacclab@utsa
STEM@UTSA
abraham@deleon
adavid@utia
alexandra@elmes
allegre@montemayor
alonzo@flowers
amanda@key
annmarie@ryan
araceli@ortiz
armando@trujillo
ashley@curves
ashley@labay
becky@huang
bedrettin@yazan
bekisizwe@ndimande
belinda@flores
betty@merchant
capri@utsa
carolina@arango-vargas
catherine@somody
claudia@garcia-louis
claudia@garcia
claudia@interiano-shiverdecker
crystal@kalinac-craig
curtis@brewer
cynthia@lima
daniela@conceicaoasilva
david@thompson
deepti@tagare
derek@robertson
devon@romero
donghun@lee
education@utsa
emily@bonner
felicia@castrovillarreal
gerald@juhnke

```

Figure 37: Entered “awk -F '@.' '{print \$1, \$2}' emails.txt > custom.txt”.



```

Kali Linux [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
Mar 26 20:50
genevevekali@syn-2603-8081-3e00-7b74-0000-0000-0000-1049: ~/Documents/EXTRACTED/username_generator
zaid.haddad
genevevekali@syn-2603-8081-3e00-7b74-0000-0000-1049:~/Documents/EXTRACTED/username_generator
$ python3 username_generator.py -w custom.txt > faculty_user_names.lst
genevevekali@syn-2603-8081-3e00-7b74-0000-0000-1049:~/Documents/EXTRACTED/username_generator
$ cat faculty_user_names.lst
coehdproposal
utsa
c.utsa
c-utsa
c_utsa
c@utsa
c!utsa
c!tsa
coehdproposalutsa
utsa@coehdproposal
coehdproposal.utsa
utsa.coehdproposal
jeff
stover
j.stover
j-stover
j_stover
j*stover
jstover
jeffstover
stoverjeff
jeff.stover
stover.jeff
limitedsubmissions
utsa
l.utsa
l-utsa
l_utsa
l!utsa
l!tsa
limitedsubmissionsutsa
utsa@limitedsubmissions
limitedsubmissions.utsa
utsa.limitedsubmissions
pacclab
utsa
p.utsa
p-utsa
p_utsa
p!utsa
putsa
pacclabutsa
utsa@pacclab
pacclab.utsa
utsa.pacclab
stem

```

Figure 38: Above is a screenshot of the usernames list created entering “python3 username_generator.py -w custom.txt > faculty_user_names.lst” - this pulls the emails from the custom.txt file.

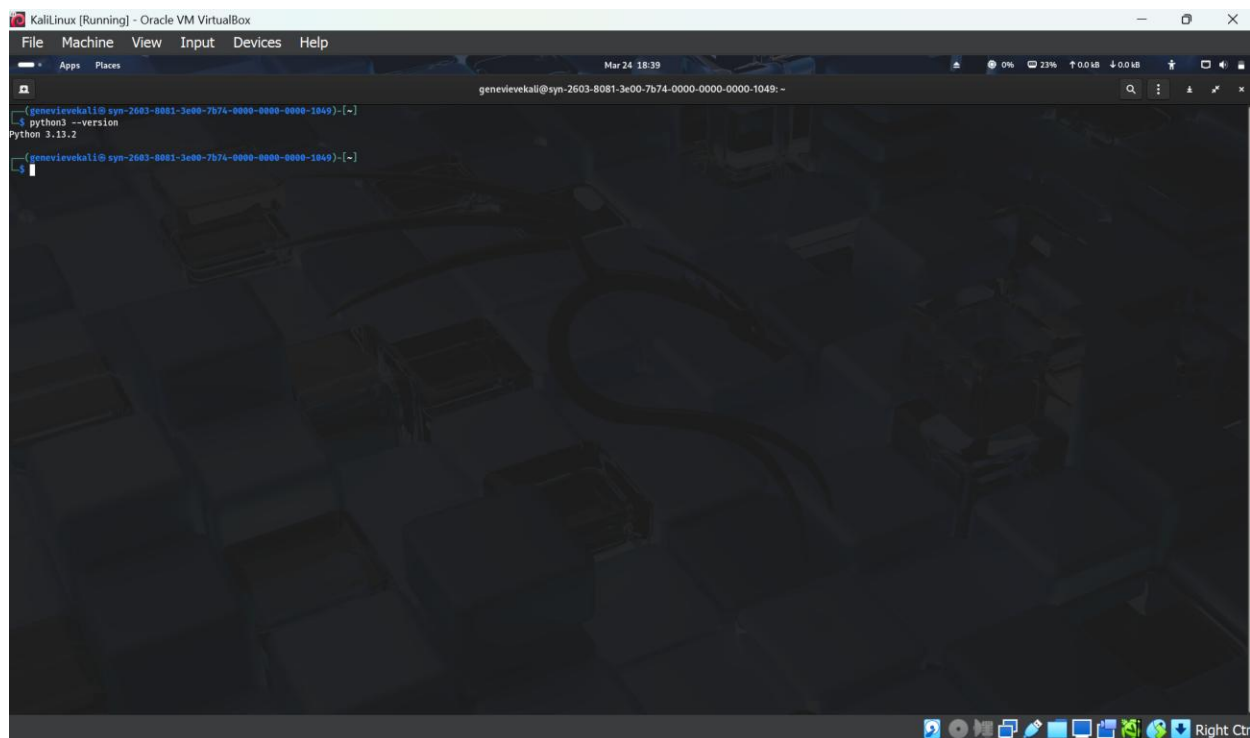


Figure 39: I entered “python3 --version” and the results showed that I am running Python 3.13.2.